

Study Guide: Introduction to Algorithms (CLRS, 4th Edition)

Generated 2026-06-03. Subject: Computer Science. Exam format: Mixed. Coverage: comprehensive. Target length: no limit.

Chapter-by-Chapter Breakdown

CLRS Introduction to Algorithms (4th Ed.) – Comprehensive Study Guide: Chapters 1–5

Ch. 1 – The Role of Algorithms in Computing

Named Entities (Terms & Definitions)

- **algorithm**: A well-defined computational procedure that takes some value(s) as input and produces some value(s) as output in a finite amount of time; a sequence of computational steps that transform input into output.
- **computational problem**: A specification of the desired input/output relationship for problem instances, typically of arbitrarily large size.
- **instance**: A particular input to a problem (satisfying the problem's constraints).
- **correct algorithm**: One that halts with the correct output for every problem instance.
- **incorrect algorithm**: Might not halt on some inputs or might produce incorrect answers; can still be useful if error rate is controllable.
- **data structure**: A way to store and organize data to facilitate access and modifications.
- **NP-complete**: A class of problems for which no efficient algorithm is known, but if an efficient algorithm exists for one, it exists for all.
- **approximation algorithm**: An algorithm that gives a good, but not necessarily optimal, solution (used for NP-complete problems).
- **online algorithm**: An algorithm that receives input over time rather than all at once.
- **parallel computer / multicore**: Computers with multiple processing cores requiring parallel algorithms.
- **keys**: The values being sorted.

- **satellite data:** Associated data that moves with the key during sorting.
- **record:** A key plus its associated satellite data.

Processes / Algorithms / Pathways

Sorting Problem Definition

- **Input:** A sequence of n numbers $a_1, a_2, \dots, a_n$
- **Output:** A permutation (reordering) $a'_1, a'_2, \dots, a'_n$ such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Insertion Sort (informal)

- Works like sorting a hand of playing cards
- Start with empty left hand, cards on table
- One at a time, take a card from the pile and insert it into correct position in left hand
- At all times, cards in left hand are sorted
- **Complexity:** $\Theta(n^2)$ worst-case, $\Theta(n)$ best-case

Merge Sort (informal)

- **Complexity:** $\Theta(n \lg n)$ worst-case
- Uses divide-and-conquer (detailed in Ch. 2)

Comparisons & Trade-offs

Dim	Insertion Sort	Merge Sort
Time (rough)	$c_1 \cdot n^2$	$c_2 \cdot n \lg n$
Constant factor	Smaller ($c_1 < c_2$)	Larger
$n=1000$	n factor	$\lg n \approx 10$
$n=1,000,000$	n factor	$\lg n \approx 20$
Cross-over	Faster for small n	Faster once n large enough

Formulas & Equations

Running time comparison (concrete example) - Computer A (10 billion instr/sec) runs insertion sort: $2n^2$ instructions - Computer B (10 million instr/sec) runs merge sort: $50 n \lg n$ instructions - For $n=10,000,000$: - Computer A: $2 \cdot (10^7)^2 / 10^9 = 20,000 \text{ sec} \approx 5.56 \text{ hours}$ - Computer B: $50 \cdot 10^7 \cdot \lg(10^7) / 10^6 \approx 50 \cdot 23.25 \approx 1163 \text{ sec} \approx 0.32 \text{ hours}$ - Computer B (slower!) runs 17x faster than Computer A

Case Studies / Classic Examples

1. **Human Genome Project:** Identifying genes, sequencing DNA base pairs, storing/analyzing data – all require sophisticated algorithms (dynamic programming Ch. 14)

2. **Internet:** Finding routes (shortest path, Ch. 22), search engines (Ch. 11, 32)
3. **Electronic commerce:** Public-key cryptography, digital signatures (Ch. 31)
4. **Manufacturing/commerce:** Linear programming for resource allocation (Ch. 29)
5. **Shortest path in road maps:** Model as graph, find shortest path (Ch. 22)
6. **Topological sorting:** Order parts so each appears before parts that use it (Ch. 20)
7. **Clustering for cancer diagnosis:** Identify tumors by similarity (Ch. 33)
8. **Data compression:** Huffman coding (Ch. 15), LZW compression
9. **Discrete Fourier Transform (FFT):** Signal processing, data compression, multiplying polynomials (Ch. 30)
10. **Traveling-salesperson problem:** NP-complete; approximation algorithms (Ch. 35)

End-of-Chapter Material

1.1-1 Real-world examples: sorting (organizing files by date); shortest distance (GPS navigation).

1.1-2 Other efficiency measures: memory usage, power consumption, communication bandwidth.

1.1-3 Example: array – strength: $O(1)$ random access; limitation: $O(n)$ insert/delete.

1.1-4 Shortest-path and traveling-salesperson are similar: both find optimal routes through a graph. Different: shortest-path can be solved efficiently ($O(E \lg V)$); traveling-salesperson is NP-complete (no known efficient algorithm).

1.1-5 Only best will do: air traffic control landing schedules. Approximately best is good enough: recommendation systems.

1.1-6 Entire input available: sorting a list of known items. Input arriving over time: stock market trading algorithms.

1.2-1 Navigation app: uses shortest-path algorithms (Dijkstra/ A^*), map rendering algorithms, address interpolation.

1.2-2 Insertion sort beats merge sort when $8n^2 < 64n \lg n \rightarrow n < 8 \lg n \rightarrow n \leq 43$. (Solve: $8n^2 < 64n \lg n \rightarrow n < 8 \lg n \rightarrow n/8 < \lg n \rightarrow n \leq 43$.)

1.2-3 $100n^2 < 2^n \rightarrow$ smallest $n = 15$. (Check: $100 \cdot 225 = 22500$ vs $2^{15} = 32768$; $n=14$: $100 \cdot 196 = 19600$ vs 16384 – insertion wins, so $n=15$.)

Problem 1-1: For each function $f(n)$ and time t , find largest n solvable. Key approach: solve $f(n) = t \times 10^6$ (since 1 microsecond). Functions: $\lg n$, $\sqrt{n}$, n , $n \lg n$, n^2 , n^3 , 2^n , $n!$.

Ch. 2 – Getting Started

Named Entities (Terms & Definitions)

- **keys:** The values being sorted.
- **satellite data:** Associated data carried with the key.
- **record:** A key plus its satellite data.
- **pseudocode:** Algorithm description similar to C/C++/Java/Python/JavaScript but using clear/concise expressions.
- **loop invariant:** A property that holds before each iteration of a loop, used to prove correctness.
- **subarray:** A contiguous portion of an array, denoted $A[i:j]$.
- **RAM (random-access machine) model:** Computation model with one processor, sequential instructions, constant-time per instruction/data access.
- **input size:** The number of items in the input (for sorting) or total bits (for integer multiplication).
- **running time:** Number of instructions and data accesses executed.
- **order of growth:** The rate of growth of running time; considers only leading term without constant coefficient.
- **Θ -notation:** Informal in Ch. 2, formalized in Ch. 3. "Roughly proportional when n is large."
- **incremental method:** Algorithm design where elements are processed one by one, inserting each into its proper place.
- **divide-and-conquer:** Algorithm design that breaks problem into subproblems, solves recursively, combines solutions.
- **recursive algorithm:** An algorithm that calls itself on smaller subproblems.
- **base case:** Smallest instance solved directly without recursion.
- **recursive case:** Case that requires recursive calls.
- **recurrence (recurrence equation):** Equation describing running time in terms of smaller inputs.

Processes / Algorithms / Pathways

INSERTION-SORT (A, n)

- **Goal:** Sort array $A[1:n]$ in monotonically increasing order.
- **Input:** Array $A[1:n]$ of n numbers.
- **Output:** $A[1:n]$ sorted.
- **Steps:**
 1. for $i = 2$ to n
 2. $key = A[i]$
 3. // Insert $A[i]$ into sorted subarray $A[1:i-1]$
 4. $j = i - 1$

5. while $j > 0$ and $A[j] > \text{key}$

6. $A[j+1] = A[j]$

7. $j = j - 1$

8. $A[j+1] = \text{key}$

- **Pseudocode:** See above.
- **Complexity:** Best-case $\Theta(n)$ (already sorted), worst-case $\Theta(n^2)$ (reverse sorted), average-case $\Theta(n^2)$.
- **Correctness proof:** Loop invariant – at start of each for loop iteration, subarray $A[1:i-1]$ contains elements originally in $A[1:i-1]$ but in sorted order.
 - **Initialization:** $i=2$, $A[1:1]$ is trivially sorted.
 - **Maintenance:** Inner while loop shifts elements right to make room; after insertion, $A[1:i]$ is sorted. Incrementing i preserves invariant.
 - **Termination:** $i=n+1$, so $A[1:n]$ is sorted.

Linear Search (Exercise 2.1-4)

- **Goal:** Find value x in array $A[1:n]$.
- **Steps:** Scan from beginning to end, compare each element to x .
- **Loop invariant:** At start of each iteration, x is not in $A[1:i-1]$.

MERGE (A, p, q, r)

- **Goal:** Merge two adjacent sorted subarrays $A[p:q]$ and $A[q+1:r]$ into sorted $A[p:r]$.
- **Input:** Array A , indices $p \leq q < r$, subarrays $A[p:q]$ and $A[q+1:r]$ sorted.
- **Output:** $A[p:r]$ sorted.
- **Steps:**
 1. $nL = q - p + 1$; $nR = r - q$
 2. Create $L[0:nL-1]$ and $R[0:nR-1]$
 3. Copy $A[p:q]$ into L , $A[q+1:r]$ into R
 4. $i=0$; $j=0$; $k=p$
 5. while $i < nL$ and $j < nR$:
 6. if $L[i] \leq R[j]$: $A[k]=L[i]$; $i++$
 7. else: $A[k]=R[j]$; $j++$
 8. $k++$
 9. Copy remainder of L or R into A
- **Complexity:** $\Theta(n)$ where $n = r-p+1$

MERGE-SORT (A, p, r)

- **Goal:** Sort subarray $A[p:r]$.

- **Steps:**
 1. if $p \geq r$: return
 2. $q = \lfloor (p+r)/2 \rfloor$
 3. MERGE-SORT(A, p, q)
 4. MERGE-SORT(A, q+1, r)
 5. MERGE(A, p, q, r)
- **Pseudocode:** See above.
- **Complexity:** $\Theta(n \lg n)$ worst-case
- **Initial call:** MERGE-SORT(A, 1, n)

BUBBLESORT (Problem 2-2)

- Repeatedly swap adjacent out-of-order elements.
- Worst-case: $\Theta(n^2)$

Selection Sort (Exercise 2.2-2)

- Repeatedly find smallest element in remaining unsorted portion and swap into position.
- Worst-case: $\Theta(n^2)$, best-case: $\Theta(n^2)$ (no improvement).

Binary Search (Exercise 2.3-6)

- On sorted array, repeatedly compare to midpoint and eliminate half.
- Worst-case: $\Theta(\lg n)$

Horner's Rule (Problem 2-3)

- Evaluate polynomial $P(x) = \sum_{i=0}^n a_i x^i$ using: $P(x) = a_n + x(a_{n-1} + x(a_{n-2} + \dots + x(a_1 + x a_0)))$
- $\Theta(n)$ time vs naive $\Theta(n^2)$

Formulas & Equations

Running time of INSERTION-SORT: - Best case: $T(n) = c_1 n + (c_2 + c_3 + c_4)(n-1) = an + b = \Theta(n)$ - Worst case: $T(n) = (c_1/2 + c_2/2 + c_3/2)n^2 + (c_4 + c_5 + c_6 + c_7/2 - c_8/2 - c_9/2 + c_{10})n - (c_{11} + c_{12} + c_{13}) = an^2 + bn + c = \Theta(n^2)$

Merge sort recurrence: - $T(n) = \Theta(1)$ if $n = 1$ - $T(n) = 2T(n/2) + \Theta(n)$ if $n > 1$ - Solution: $T(n) = \Theta(n \lg n)$ - (With detailed form: $T(n) = 2T(n/2) + c_1 n$, base $T(1) = c_2$) - Recursion tree: $\lg n + 1$ levels; each level costs $c_1 n$; leaf level costs $c_2 n$; total = $c_1 n \lg n + c_2 n = \Theta(n \lg n)$

General divide-and-conquer recurrence: - $T(n) = \Theta(1)$ if $n < n_0$ (base case) - $T(n) = aT(n/b) + D(n) + C(n)$ if $n \geq n_0$ - Where a = number of subproblems, n/b = size of each, $D(n)$ = divide time, $C(n)$ = combine time

Factorial: $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

Comparisons & Trade-offs

Dim	Insertion Sort	Merge Sort
Worst-case	$\Theta(n^2)$	$\Theta(n \lg n)$
Best-case	$\Theta(n)$	$\Theta(n \lg n)$
In-place	Yes	No (uses extra arrays)
Method	Incremental	Divide-and-conquer
Small inputs	Faster (lower constants)	Slower

Design Paradigms

- **Incremental method:** Insertion sort – process elements one at a time, maintaining sorted prefix.
- **Divide-and-conquer:** Merge sort – divide into subproblems, solve recursively, combine.

Edge Cases / Traps

- **Already sorted array:** Insertion sort runs in $\Theta(n)$ (best case); merge sort still runs in $\Theta(n \lg n)$.
- **Reverse sorted array:** Insertion sort worst-case $\Theta(n^2)$.
- **Single element:** Already sorted; base case of recursion.
- **Empty subarray:** If $p > r$, subarray is empty – MERGE-SORT test “if $p \geq r$ ” handles this.
- **Off-by-one:** In MERGE, subarray $A[p:q]$ has length $q-p+1$; $A[q+1:r]$ has length $r-q$.

End-of-Chapter Material

2.1-1 Insertion sort on $\langle 31, 41, 59, 26, 41, 58 \rangle$: - $i=2$: key=41, compare $31 \leq 41 \rightarrow \langle 31, 41, 59, 26, 41, 58 \rangle$ - $i=3$: key=59, compare $41 \leq 59 \rightarrow \langle 31, 41, 59, 26, 41, 58 \rangle$ - $i=4$: key=26, shift 59, 41, 31 $\rightarrow \langle 26, 31, 41, 59, 41, 58 \rangle$ - $i=5$: key=41, shift 59 $\rightarrow \langle 26, 31, 41, 41, 59, 58 \rangle$ - $i=6$: key=58, shift 59 $\rightarrow \langle 26, 31, 41, 41, 58, 59 \rangle$

2.1-2 SUM-ARRAY: Loop invariant – at start of each iteration i , $\text{sum} = \text{sum of } A[1:i-1]$. Initialization: $\text{sum}=0$, $i=1$, $A[1:0]$ empty $\text{sum}=0$. Maintenance: $\text{sum} += A[i]$. Termination: $i=n+1$, $\text{sum} = \text{sum of all elements}$.

2.1-3 Decreasing insertion sort: change line 5 to $\text{while } j > 0 \text{ and } A[j] < \text{key}$.

2.1-4 Linear search pseudocode:

```
LINEAR-SEARCH(A, n, x)
1 for i = 1 to n
2     if A[i] == x
3         return i
4 return NIL
```

Loop invariant: $A[1:i-1]$ does not contain x .

2.1-5 Binary addition:

```
ADD-BINARY-INTEGERS(A, B, n)
1 carry = 0
2 for i = 0 to n-1
3     sum = A[i] + B[i] + carry
4     C[i] = sum mod 2
5     carry = sum div 2
6 C[n] = carry
7 return C
```

$$2.2-1 \quad n^3/1000 + 100n^2 - 100n + 3 = \Theta(n^3)$$

2.2-2 Selection sort:

```
SELECTION-SORT(A, n)
1 for i = 1 to n-1
2     min = i
3     for j = i+1 to n
4         if A[j] < A[min]
5             min = j
6     swap A[i] with A[min]
```

Only $n-1$ elements needed because after placing first $n-1$, the last element is in correct position. Worst-case $\Theta(n^2)$. Best-case $\Theta(n^2)$ – no improvement.

2.2-3 Linear search: average-case: $(n+1)/2$ elements checked = $\Theta(n)$. Worst-case: n elements = $\Theta(n)$.

2.2-4 Check if already sorted at start; if so, output immediately ($\Theta(n)$ best-case).

2.3-1 Merge sort on $\langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$: - Divide:
 $[3, 41, 52, 26]$ $[38, 57, 9, 49]$ - Divide: $[3, 41]$ $[52, 26]$ $[38, 57]$ $[9, 49]$
- Divide: $[3]$ $[41]$ $[52]$ $[26]$ $[38]$ $[57]$ $[9]$ $[49]$ - Merge: $[3, 41]$
 $[26, 52]$ $[38, 57]$ $[9, 49]$ - Merge: $[3, 26, 41, 52]$ $[9, 38, 49, 57]$ -
Merge: $[3, 9, 26, 38, 41, 49, 52, 57]$

2.3-2 Test “if $p \geq r$ ” vs “if $p \neq r$ ”: If called initially with $n \geq 1$, p starts at 1, $r=n$. Since $q = \lfloor (p+r)/2 \rfloor$, we have $p \leq q < r$, so recursive calls have $p \leq q$ and $q+1 \leq r$. The subarrays always have at least one element, so $p > r$ never occurs.

2.3-3 Loop invariant for MERGE while loop (lines 12-18): At start of each iteration, subarray $A[p:k-1]$ contains the $k-p$ smallest elements of $L[0:n_L-1]$ and $R[0:n_R-1]$ in sorted order, and $L[i]$ and $R[j]$ are the smallest elements of their respective arrays not yet copied back.

2.3-4 Prove $T(n) = n \lg n$ for recurrence $T(n) = 2T(n/2) + n$, $T(1) = 0$ (or $\Theta(1)$). Base: $n=1$, $T(1) = 0 = 1 \cdot \lg 1$. Inductive step: $T(2n) = 2T(n)$

$+2n=2(n \lg n)+2n=2n(\lg n+1)=2n \lg(2n)$. (If using constant base c , show $T(n)=c \cdot n + c \cdot n \lg n$.)

2.3-5 Recursive insertion sort:

```

RECURSIVE-INSERTION-SORT(A, n)
1 if n > 1
2   RECURSIVE-INSERTION-SORT(A, n-1)
3   key = A[n]
4   j = n-1
5   while j > 0 and A[j] > key
6     A[j+1] = A[j]
7     j = j-1
8   A[j+1] = key

```

Recurrence: $T(n) = T(n-1) + \theta(n) \rightarrow T(n) = \theta(n^2)$

2.3-6 Binary search:

```

BINARY-SEARCH(A, n, v)
1 lo = 1; hi = n
2 while lo ≤ hi
3   mid = ⌊(lo+hi)/2⌋
4   if A[mid] == v: return mid
5   if A[mid] < v: lo = mid+1
6   else: hi = mid-1
7 return NIL

```

Worst-case: $\theta(\lg n)$. Using binary search in insertion sort does NOT improve to $\theta(n \lg n)$ because shifting elements still takes $\theta(n)$ per element.

2.3-7 Two-sum algorithm:

```

TWO-SUM(S, n, x)
1 MERGE-SORT(S, n) //  $\theta(n \lg n)$ 
2 for i = 1 to n
3   if BINARY-SEARCH(S, n, x-S[i]) returns index j where j ≠ i
4     return (i, j)
5 return NIL

```

Total: $\theta(n \lg n)$.

Problem 2-1 (Insertion sort on small arrays in merge sort): a. Insertion sort on n/k lists of size k : each list $\theta(k^2)$, n/k lists $\rightarrow \theta(nk)$. b. Merging n/k lists: merge tree has $\lg(n/k)$ levels, each level $\theta(n) \rightarrow \theta(n \lg(n/k))$. c. For same running time as standard merge sort: $\theta(nk + n \lg(n/k)) = \theta(n \lg n) \rightarrow k = \theta(\lg n)$. d. In practice, choose k where insertion sort outperforms merge sort on small arrays (typically 10-50).

Problem 2-2 (Bubblesort): a. BUBBLESORT terminates with $A'[1] \leq A'[2] \leq \dots \leq A'[n]$. Need to prove A' is a permutation of original A . b. Inner loop invariant: After inner loop iteration j , $A[j]$ is

the minimum of $A[j:n]$ (for descending j). c. Outer loop invariant: After $i-1$ iterations, $A[1:i-1]$ contains the $i-1$ smallest elements in sorted order. d. Worst-case: $\Theta(n^2)$. Comparable to insertion sort but with more swaps on average.

Problem 2-3 (Horner's rule): a. $\Theta(n)$. b. Naive: for $i=0..n$, compute $\text{term} = a_i$; for $j=1..i$, $\text{term} *= x$; $\text{sum} += \text{term}$. $\Theta(n^2)$. c. Loop invariant: At start of iteration i , $p = \sum_{k=0}^{i-1} a_{k+i} x^{n-k}$. At termination $i=n$ (or $i=0$ before loop), $p = \sum_{k=0}^n a_k x^{n-k} = P(x)$.

Problem 2-4 (Inversions): a. Inversions in $[2, 3, 8, 6, 1]$: $(1, 5)$, $(2, 5)$, $(3, 4)$, $(3, 5)$, $(4, 5)$ – 5 inversions. b. Array $[n, n-1, \dots, 1]$ has most inversions: $n(n-1)/2$. c. Running time of insertion sort = $\Theta(n + d)$ where d = number of inversions. Each swap eliminates one inversion. d. Count inversions by modifying merge sort: during merge, when $R[j] < L[i]$, add $(nL - i)$ inversions. $\Theta(n \lg n)$.

Ch. 3 – Characterizing Running Times

Named Entities (Terms & Definitions)

- **asymptotic efficiency:** How running time increases with input size in the limit as input size grows without bound.
- **O -notation (big-oh):** Asymptotic upper bound. $f(n) = O(g(n))$ means $f(n)$ grows no faster than $g(n)$ (to within constant factor).
- **Ω -notation (big-omega):** Asymptotic lower bound. $f(n)$ grows at least as fast as $g(n)$.
- **Θ -notation (theta):** Asymptotically tight bound. $f(n)$ grows at exactly the rate of $g(n)$ (to within constant factors).
- **o -notation (little-oh):** Upper bound that is NOT asymptotically tight.
- **ω -notation (little-omega):** Lower bound that is NOT asymptotically tight.
- **asymptotically nonnegative:** $f(n) \geq 0$ for all sufficiently large n .
- **asymptotically positive:** $f(n) > 0$ for all sufficiently large n .
- **asymptotically smaller/larger:** f is asymptotically smaller than g if $f = o(g)$; larger if $f = \omega(g)$.
- **monotonically increasing:** $m \leq n \implies f(m) \leq f(n)$.
- **monotonically decreasing:** $m \leq n \implies f(m) \geq f(n)$.
- **strictly increasing:** $m < n \implies f(m) < f(n)$.
- **floor ($\lfloor x \rfloor$):** Greatest integer $\leq x$.
- **ceiling ($\lceil x \rceil$):** Least integer $\geq x$.
- **modular arithmetic:** $a \bmod n$ = remainder of a/n ; $a \equiv b \pmod{n}$ if $(a \bmod n) = (b \bmod n)$.
- **polynomial:** $p(n) = \sum_{d=0}^D a_d n^d$, with $a_D \neq 0$; asymptotically positive if $a_D > 0$.

- **polynomially bounded:** $f(n) = O(n^k)$ for some constant k .
- **polylogarithmically bounded:** $f(n) = O(\lg^k n)$ for some constant k .
- **exponential:** Function of form a^n with $a > 1$.
- **factorial ($n!$):** $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$.
- **iterated logarithm ($\lg^* n$):** Number of times $\lg$ must be applied to reduce n to ≤ 1 .
- **Fibonacci numbers:** $F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$ for $i \geq 2$.
- **golden ratio (ϕ):** $\phi = (1 + \sqrt{5})/2 \approx 1.618$; $\bar{\phi} = (1 - \sqrt{5})/2 \approx -0.618$.
- **Stirling's approximation:** $n! = \sqrt{2\pi n} (n/e)^n (1 + O(1/n))$.
- **watershed function:** $n^{\lceil \log_b a \rceil}$ in master theorem.

Formulas & Equations

O -notation (formal)

$O(g(n)) = \{f(n) : \exists \text{ positive constants } c, n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}$

Ω -notation (formal)

$\Omega(g(n)) = \{f(n) : \exists \text{ positive constants } c, n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}$

Θ -notation (formal)

$\Theta(g(n)) = \{f(n) : \exists \text{ positive constants } c_1, c_2, n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}$

o -notation (formal)

$o(g(n)) = \{f(n) : \exists \text{ positive constants } c > 0, \exists n_0 > 0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}$

ω -notation (formal)

$\omega(g(n)) = \{f(n) : \exists \text{ positive constants } c > 0, \exists n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}$

Theorem 3.1

$f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

Limit characterizations

- $f(n) = o(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$
- $f(n) = \omega(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$

Transitivity

- $f = \Theta(g)$ and $g = \Theta(h) \Rightarrow f = \Theta(h)$
- $f = O(g)$ and $g = O(h) \Rightarrow f = O(h)$
- $f = \Omega(g)$ and $g = \Omega(h) \Rightarrow f = \Omega(h)$

Reflexivity

$f = \Theta(f)$, $f = O(f)$, $f = \Omega(f)$

Symmetry

$f = \Theta(g)$ iff $g = \Theta(f)$

Transpose symmetry

- $f = O(g)$ iff $g = \Omega(f)$
- $f = o(g)$ iff $g = \omega(f)$

Floor/ceiling properties

- $\lfloor x \rfloor - 1 < x \leq \lfloor x \rfloor + 1$
- $\lceil x \rceil - 1 < x \leq \lceil x \rceil + 1$
- $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$ for integer n
- For real $x \geq 0$ and integers $a, b > 0$: $\lfloor \lfloor x/a \rfloor / b \rfloor = \lfloor x/(ab) \rfloor$
and $\lceil \lceil x/a \rceil / b \rceil = \lceil x/(ab) \rceil$

Polynomials

- $p(n) = \sum_{d=0}^D a_d n^d$ with $a_D > 0 \Rightarrow p(n) = \Theta(n^D)$

Exponentials

- $a^0 = 1$, $a^1 = a$, $a^{-1} = 1/a$
- $(a^m)^n = a^{mn}$
- $a^m a^n = a^{m+n}$
- For $a > 1$, b : $\lim_{n \rightarrow \infty} n^b / a^n = 0$ (exponential dominates polynomial)
- $e = 2.71828\dots$; $e^x = \sum_{k=0}^{\infty} x^k / k! \geq 1+x$ (equality only at $x=0$)
- For $|x| \leq 1$: $1 + x \leq e^x \leq 1 + x + x^2$
- As $x \rightarrow 0$: $e^x = 1 + x + \Theta(x^2)$

Logarithms

- $\lg n = \log_2 n$ (binary), $\ln n = \log_e n$ (natural)
- $\lg^k n = (\lg n)^k$, $\lg \lg n = \lg(\lg n)$
- $\log_b n$ undefined for $n \leq 0$; strictly increasing for $n > 0$
- $a = b^{\log_b a}$, $\log_c(ab) = \log_c a + \log_c b$
- $\log_b a^n = n \log_b a$
- $\log_b a = \log_c a / \log_c b$ (change of base)

- $\log_b(1/a) = -\log_b a$
- $a^{\{\log_b c\}} = c^{\{\log_b a\}}$
- For $x > -1$: $x/(1+x) \leq \ln(1+x) \leq x$ (equality only at $x=0$)
- $\ln(1+x) = x - x^2/2 + x^3/3 - x^4/4 + \dots$
- For any constants $a>0$, b : $\lim_{n \rightarrow \infty} \lg^b n / n^a = 0$
(polynomial dominates polylogarithm)

Factorial

- $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$, with $0! = 1$
- $n! \leq n^n$ (weak bound)
- Stirling's approximation: $n! = \sqrt{2\pi n} (n/e)^n (1 + \Theta(1/n))$
- $n! = o(n^n)$, $n! = \omega(2^n)$, $\lg(n!) = \Theta(n \lg n)$

Iterated logarithm ($\lg^* n$)

- $\lg^* n = \min\{i \geq 0: \lg^{(i)} n \leq 1\}$
- $\lg^* 2 = 1$, $\lg^* 4 = 2$, $\lg^* 16 = 3$, $\lg^* 65536 = 4$, $\lg^* 2^{65536} = 5$
- Extremely slow-growing; rarely exceeds 5 for practical inputs.

Fibonacci numbers

- $F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$ for $i \geq 2$
- $\phi^2 = \phi + 1$ (golden ratio equation)
- $\phi = (1+\sqrt{5})/2$, $\bar{\phi} = (1-\sqrt{5})/2$
- $F_i = (\phi^i - \bar{\phi}^i)/\sqrt{5}$
- $F_i = \lceil \phi^i / \sqrt{5} + 1/2 \rceil$ (nearest integer to $\phi^i / \sqrt{5}$)
- Fibonacci numbers grow exponentially.

Comparisons (Analogy with Real Numbers)

Asymptotic Real Numbers

$$f = O(g) \quad a \leq b$$

$$f = \Omega(g) \quad a \geq b$$

$$f = \Theta(g) \quad a = b$$

$$f = o(g) \quad a < b$$

$$f = \omega(g) \quad a > b$$

Note: Trichotomy does NOT hold for asymptotic notation – not all functions are asymptotically comparable (e.g., n vs $n^{\{1+\sin n\}}$).

Edge Cases / Traps

- **$\Theta(n^2) \neq O(n^2)$:** Θ is tight bound; O is only upper bound.
"Insertion sort runs in $\Theta(n^2)$ " is wrong as blanket statement (best case is $\Theta(n)$).
- **Asymptotically nonnegative:** Required for all notations;
 $O(g(n))$ is empty if g negative.

- **o vs O:** o is strictly looser bound than O . $2n = O(n^2)$ but also $2n = o(n^2)$; $2n^2 \neq o(n^2)$.
- **Gap between cases:** Master theorem has gaps (e.g., $f(n) = n/\lg n$ falls in gap between cases 1 and 2).
- **Anonymous functions:** In $2n^2 + \theta(n) = \theta(n^2)$, the $\theta(n)$ on LHS and $\theta(n^2)$ on RHS represent (possibly different) anonymous functions.
- **Constants hidden:** When using substitution method, must use explicit constants, not asymptotic notation in inductive hypothesis.

End-of-Chapter Material

3.1-1 Modify lower bound for non-multiple-of-3: use $\lceil n/3 \rceil$ largest values in first $\lceil n/3 \rceil$ positions. The middle $\lceil n/3 \rceil$ positions are passed through by at least $\lceil n/3 \rceil$ values, giving $\Omega(n^2)$.

3.1-2 Selection sort: $\theta(n^2)$ – doubly nested loops, each roughly n iterations.

3.1-3 Generalization with α : αn largest values start in first αn positions. Each must pass through middle $(1-2\alpha)n$ positions. Need $\alpha < 1/2$ for middle to be non-empty. The number of passes = $\alpha(1-2\alpha)n^2$. Maximized when $\alpha = 1/4$.

3.2-1 $\max(f, g) \leq f+g$, so $\max(f, g) = O(f+g)$. Also $\max(f, g) \geq (f+g)/2$ (since both nonnegative), so $\max(f, g) = \Omega(f+g)$. Thus $\max(f, g) = \theta(f+g)$.

3.2-2 “Running time is at least $O(n^2)$ ” is meaningless because O is an upper bound, not lower. “At least” suggests a lower bound, conflicting with O ’s upper bound meaning.

3.2-3 $2^{n+1} = 2 \cdot 2^n = O(2^n)$. $2^{2n} = (2^n)^2 \neq O(2^n)$ since ratio is 2^n which is unbounded.

3.2-4 Proof of Theorem 3.1: (i) If $f = \theta(g)$, then $\exists c_1, c_2, n_1$: $c_1 g \leq f \leq c_2 g$. Then $f \leq c_2 g$ so $f = O(g)$; $c_1 g \leq f$ so $f = \Omega(g)$. (ii) If $f = O(g)$ and $f = \Omega(g)$, then $\exists c_1, n_1$: $f \leq c_1 g$ for $n \geq n_1$ and $\exists c_2, n_2$: $c_2 g \leq f$ for $n \geq n_2$. Let $n_3 = \max(n_1, n_2)$. Then $c_2 g \leq f \leq c_1 g$, so $f = \theta(g)$.

3.2-5 Running time is $\theta(g(n))$ iff worst-case = $O(g(n))$ and best-case = $\Omega(g(n))$. Proof: $T_{\text{worst}}(n) = \max_{\text{input of size } n} T(n)$, $T_{\text{best}}(n) = \min_{\text{input of size } n} T(n)$. If $T(n) = \theta(g(n))$ then $\exists$ inputs: $c_1 g \leq T \leq c_2 g$, so $T_{\text{worst}} = O(g)$, $T_{\text{best}} = \Omega(g)$. Conversely, if $T_{\text{worst}} = O(g)$ and $T_{\text{best}} = \Omega(g)$, then $\exists c_1, c_2$: $T_{\text{best}} \geq c_1 g$ and $T_{\text{worst}} \leq c_2 g$, so $c_1 g \leq T_{\text{best}} \leq T(n) \leq T_{\text{worst}} \leq c_2 g$, hence $T(n) = \theta(g(n))$.

3.2-6 $o(g) \cap \omega(g) = \emptyset$. If $f \in o(g)$, then $\lim f/g = 0$. If $f \in \omega(g)$, then $\lim f/g = \infty$. Both cannot hold simultaneously.

3.2-7 $\Omega(g(n, m)) = \{f(n, m): \exists c, n_0, m_0: 0 \leq c \cdot g(n, m) \leq f(n, m) \text{ for all } n \geq n_0 \text{ or } m \geq m_0\}$. $\theta(g(n, m)) = \{f(n, m): \exists c_1, c_2, n_0, m_0: 0 \leq c_1 \cdot g(n, m) \leq f(n, m) \leq c_2 \cdot g(n, m) \text{ for all } n \geq n_0 \text{ or } m \geq m_0\}$.

3.3-1 If f, g monotonically increasing: $(f+g)(m) = f(m)+g(m) \leq f(n)+g(n) = (f+g)(n)$. $(f \circ g)(m) = f(g(m)) \leq f(g(n)) = (f \circ g)(n)$. If nonnegative: $(f \cdot g)(m) \leq (f \cdot g)(n)$.

3.3-2 $\lceil \alpha n \rceil + \lceil (1-\alpha)n \rceil = n$. Let $\alpha n = \lceil \alpha n \rceil + r$ where $0 \leq r < 1$. Then $(1-\alpha)n = n - \alpha n = n - \lceil \alpha n \rceil - r$. So $\lceil (1-\alpha)n \rceil = n - \lceil \alpha n \rceil$ (since $-r$ makes it ceiling up). Sum = n .

3.3-3 $(n+o(n))^k = n^k + k \cdot n^{k-1} \cdot o(n) + \dots = n^k + o(n^k) = \Theta(n^k)$. Hence $\lceil n \rceil^k = \Theta(n^k)$ and $\lfloor n \rfloor^k = \Theta(n^k)$.

3.3-4 Equation (3.21): $a^{\lceil \log_b c \rceil} = c^{\lceil \log_b a \rceil}$. Take logs: $\log_b(a^{\lceil \log_b c \rceil}) = \log_b c \cdot \log_b a$; $\log_b(c^{\lceil \log_b a \rceil}) = \log_b a \cdot \log_b c$. Equal. Equations (3.26)-(3.28): Stirling: $n! = \sqrt{2\pi n} (n/e)^{n(1+O(1/n))}$. $\lg(n!) = \Theta(n \lg n)$.

3.3-5 $\lg n!$ is NOT polynomially bounded (grows faster than any polynomial). $\lg \lg n!$ IS polynomially bounded (it's $O(\lg n)$ which is $O(n^\epsilon)$ for any $\epsilon > 0$).

3.3-6 $\lg(\lg^* n)$ and $\lg(\lg n)$: Let $m = \lg n$. Then $\lg(\lg n) = \lg^* m$. And $\lg(\lg^* n) = \lg(\lg^* 2^m)$. Since $\lg^* 2^m = \lg^* m + 1$, we have $\lg(\lg^* n) = \lg(\lg^* m + 1) \approx \lg(\lg^* m)$. Compare: $\lg(\lg^* n)$ grows as $\lg(\lg^* n)$ while $\lg(\lg n)$ grows as $\lg m$. $\lg^*$ grows much faster than $\lg$, so $\lg(\lg n) = \lg m$ is asymptotically larger than $\lg(\lg^* n) \approx \lg(\lg^* m)$.

3.3-7 $\phi = (1+\sqrt{5})/2$ and $\psi = (1-\sqrt{5})/2$: $\phi^2 = (3+\sqrt{5})/2 = (1+\sqrt{5})/2 + 1 = \phi + 1$. Similarly $\psi^2 = \psi + 1$.

3.3-8 $F_i = (\phi^i - \psi^i)/\sqrt{5}$. Base: $i=0$: $(1-1)/\sqrt{5}=0$; $i=1$: $(\phi-\psi)/\sqrt{5} = (\sqrt{5})/\sqrt{5} = 1$. Inductive: assume for $i-1, i$. $F_{i+1} = F_i + F_{i-1} = (\phi^i - \psi^i)/\sqrt{5} + (\phi^{i-1} - \psi^{i-1})/\sqrt{5} = (\phi^i + \phi^{i-1} - \psi^i - \psi^{i-1})/\sqrt{5} = (\phi^{i-1}(\phi+1) - \psi^{i-1}(\psi+1))/\sqrt{5} = (\phi^{i-1}\phi^2 - \psi^{i-1}\psi^2)/\sqrt{5} = (\phi^{i+1} - \psi^{i+1})/\sqrt{5}$.

3.3-9 $k \lg k = \Theta(n)$ implies $k = \Theta(n/\lg n)$. Suppose $k = cn/\lg n$ for some c . Then $k \lg k = (cn/\lg n) \cdot \lg(cn/\lg n) = (cn/\lg n)(\lg n + \lg c - \lg \lg n) = cn(1 + o(1)) = \Theta(n)$. So $n = \Theta(k \lg k)$ $\Rightarrow k = \Theta(n/\lg n)$.

Ch. 4 – Divide-and-Conquer

Named Entities (Terms & Definitions)

- **recurrence**: An equation describing a function in terms of its value on smaller arguments.
- **algorithmic recurrence**: A recurrence where for all $n < n_0$, $T(n) = \Theta(1)$, and every recursion path terminates in finite steps.
- **substitution method**: Guess form of solution, prove by induction.

- **recursion-tree method**: Model recurrence as tree, sum costs per level.
- **master method**: Cookbook method for recurrences of form $T(n) = aT(n/b) + f(n)$.
- **Akra-Bazzi method**: General method for recurrences with differently-sized subproblems.
- **driving function ($f(n)$)**: The non-recursive part of a master recurrence.
- **watershed function ($n^{\log_b a}$)**: The function compared against $f(n)$ in master theorem.
- **regularity condition**: $af(n/b) \leq cf(n)$ for some $c < 1$ (required for master theorem case 3).
- **polynomial-growth condition**: $f(\theta(n)) = \theta(f(n))$ in a strong sense; required for ignoring floors/ceilings in Akra-Bazzi recurrences.
- **Monge array**: Array where $A[i,j] + A[k,l] \leq A[i,l] + A[k,j]$ for all $i < k, j < l$.
- **dense matrix**: Matrix where most entries are nonzero.
- **sparse matrix**: Matrix where most entries are zero.

Processes / Algorithms / Pathways

MATRIX-MULTIPLY (A, B, C, n) – naive

```

MATRIX-MULTIPLY(A, B, C, n)
1 for i = 1 to n
2   for j = 1 to n
3     for k = 1 to n
4       c_ij = c_ij + a_ik · b_kj

```

- **Complexity**: $\Theta(n^3)$

MATRIX-MULTIPLY-RECURSIVE (A, B, C, n)

```

MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)
1 if n == 1
2   c[] = a[] · b[]
3   return
4 partition A, B, C into n/2×n/2 submatrices
5 MATRIX-MULTIPLY-RECURSIVE(A11, B11, C11, n/2)
6 MATRIX-MULTIPLY-RECURSIVE(A12, B11, C12, n/2)
7 MATRIX-MULTIPLY-RECURSIVE(A11, B12, C11, n/2)
8 MATRIX-MULTIPLY-RECURSIVE(A12, B12, C12, n/2)
9 MATRIX-MULTIPLY-RECURSIVE(A21, B11, C21, n/2)
10 MATRIX-MULTIPLY-RECURSIVE(A22, B11, C22, n/2)
11 MATRIX-MULTIPLY-RECURSIVE(A21, B12, C21, n/2)
12 MATRIX-MULTIPLY-RECURSIVE(A22, B12, C22, n/2)

```

- **Recurrence**: $T(n) = 8T(n/2) + \Theta(1)$
- **Solution**: $T(n) = \Theta(n^3)$ (by master theorem case 1)
- **Note**: The partitioning equations are:
 - $C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$
 - $C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$

- $C_{ij} = A_{ij} \cdot B_{ij} + A_{ij} \cdot B_{ij}$
- $C_{ij} = A_{ij} \cdot B_{ij} + A_{ij} \cdot B_{ij}$

Strassen's Algorithm

- **Goal:** Multiply $n \times n$ matrices in $O(n^3)$ time.
- **Key insight:** Only 7 recursive multiplications instead of 8, at cost of 18 matrix additions/subtractions.

Step 1: Partition into $n/2 \times n/2$ submatrices ($O(1)$ using index calculations).

Step 2: Create 10 sum/difference matrices $S_1, \dots, S_{10}$ ($O(n^2)$):

- $S_1 = B_{11} - B_{21}$
- $S_2 = A_{11} + A_{21}$
- $S_3 = A_{11} + A_{12}$
- $S_4 = B_{12} - B_{22}$
- $S_5 = A_{21} + A_{22}$
- $S_6 = B_{21} + B_{22}$
- $S_7 = A_{12} - A_{22}$
- $S_8 = B_{11} + B_{12}$
- $S_9 = A_{11} - A_{21}$
- $S_{10} = B_{21} + B_{22}$

Step 3: Recursively compute 7 products $P_1, \dots, P_7$ ($7T(n/2)$):

- $P_1 = A_{11} \cdot S_1$
- $P_2 = A_{11} \cdot B_{12} - A_{21} \cdot B_{22} - P_1$
- $P_3 = S_2 \cdot B_{11} = A_{11} \cdot B_{11} + A_{21} \cdot B_{11} - P_1$
- $P_4 = A_{12} \cdot S_1 = A_{12} \cdot B_{11} - A_{22} \cdot B_{21} - P_1$
- $P_5 = S_3 \cdot S_4 = A_{11} \cdot B_{12} + A_{12} \cdot B_{21} + A_{21} \cdot B_{12} + A_{22} \cdot B_{21} - P_1$
- $P_6 = S_5 \cdot S_6 = A_{21} \cdot B_{11} + A_{22} \cdot B_{12} - A_{11} \cdot B_{21} - A_{12} \cdot B_{22} - P_1$
- $P_7 = S_7 \cdot S_{10} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22} - A_{21} \cdot B_{11} - A_{22} \cdot B_{22} - P_1$

Step 4: Combine results ($O(n^2)$ with 12 matrix additions/subtractions):

- $C_{11} = C_{11} + P_2 + P_3 - P_4 + P_5 - C_{11} = C_{11} + P_2 + P_3 - P_4 + P_5$
- $C_{12} = C_{12} + P_6 + P_7 - P_4 - P_5 - P_6$
- $C_{21} = C_{21} + P_2 + P_3 - P_4 - P_5 - P_6$
- $C_{22} = C_{22} + P_2 + P_3 - P_4 - P_5 - P_6$

- **Recurrence:** $T(n) = 7T(n/2) + O(n^2)$
- **Solution:** $T(n) = O(n^{\lceil \lg 7 \rceil}) = O(n^{2.81})$ (by master theorem case 1)

Formulas & Equations

General divide-and-conquer recurrence

$$T(n) = aT(n/b) + f(n)$$

Master Theorem (Theorem 4.1)

Let $a > 0$, $b > 1$ constants; $f(n)$ driving function. $T(n) = aT(n/b) + f(n)$ (with implicit floors/ceilings).

- **Case 1:** if $\epsilon > 0$: $f(n) = O(n^{\log_b a - \epsilon})$ $\Rightarrow T(n) = O(n^{\log_b a})$
- **Case 2:** if $k \geq 0$: $f(n) = \Theta(n^{\log_b a} \lg^k n)$ $\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
- **Case 3:** if $\epsilon > 0$: $f(n) = \Omega(n^{\log_b a + \epsilon})$ AND regularity condition $af(n/b) \leq cf(n)$ for some $c < 1$ $\Rightarrow T(n) = O(f(n))$

Akra-Bazzi method

For $T(n) = \sum_{i=1}^k a_i T(n/b_i) + f(n)$: Find p such that $\sum a_i b_i^{-p} = 1$. Then $T(n) = \Theta(n^p(1 + \int_1^n f(u)/u^{p+1} du))$

Example recurrences and solutions

Recurrence	a	b	f(n)	$n^{\{\log_b a\}}$	Case	Solution
Merge sort: $T(n) = 2T(n/2) + n$	2	2	n	n	2 ($k=0$)	$\Theta(n \lg n)$
Matrix recursive: $T(n) = 8T(n/2) + 1$	8	2	$\Theta(1)$	n^3	1	$\Theta(n^3)$
Strassen: $T(n) = 7T(n/2) + n^2$	7	2	n^2	$n^{\{\lg 7\}} \approx n^{\{2.81\}}$	1	$\Theta(n^{\{\lg 7\}})$
$T(n) = 9T(n/3) + n$	9	3	n	n^2	1	$\Theta(n^2)$
$T(n) = T(2n/3) + 1$	1	3/2	1	$n^{\theta=1}$	2	$\Theta(\lg n)$
$T(n) = 3T(n/4) + n \lg n$	3	4	$n \lg n$	$n^{\{\log_4 3\}} \approx n^{\{0.79\}}$	3	$\Theta(n \lg n)$
$T(n) = 2T(n/2) + n \lg n$	2	2	$n \lg n$	n	2 ($k=1$)	$\Theta(n \lg^2 n)$

Akra-Bazzi example

$T(n) = T(n/5) + T(7n/10) + n$ (from selection algorithm, Ch. 9)
Find p : $(1/5)^p + (7/10)^p = 1$. Since $p \in (0,1)$, solution $T(n) = \Theta(n)$.

Comparisons & Trade-offs

Algorithm	Recursions	Combine cost	Recurrence	Solution
Naive multiply	8 subproblems	$\Theta(1)$	$8T(n/2) + \Theta(1)$	$\Theta(n^3)$
Strassen	7 subproblems	$\Theta(n^2)$	$7T(n/2) + \Theta(n^2)$	$\Theta(n^{\{\lg 7\}}) = \Theta(n^{\{2.81\}})$

Edge Cases / Traps

- **Master theorem gaps:** $f(n) = n/\lg n$ falls between cases 1 and 2 (not polynomially slower than n). $f(n) = n \lg n$ falls between cases 2 and 3 when comparing with $n^{\{\log_b a\}}$ if $\lg n$ is in the gap – actually case 2 does cover it with $k=1$.
- **Regularity condition:** Must hold for case 3. Counterexample: $f(n) = 2^{\{\lg n\}}$ satisfies $f(n) = \Omega(n^{\{\log_b a + \epsilon\}})$ but fails regularity.
- **Floors and ceilings:** Can generally be ignored for master recurrences and for Akra-Bazzi recurrences whose driving function satisfies the polynomial-growth condition.

- **Algorithmic recurrence convention:** Base cases $T(n) = \Theta(1)$ for $n < n_0$ are implicit; asymptotic solution doesn't depend on choice of n_0 .
- **Inequality recurrences:** $T(n) \leq 2T(n/2) + n \Rightarrow O(n \lg n)$; $T(n) \geq 2T(n/2) + n \Rightarrow \Omega(n \lg n)$.

End-of-Chapter Material

4.1-1 Generalize for non-power-of-2: use ceil/floor for $n/2$.
 Recurrence: $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \dots$ Still $\Theta(n^3)$ by master theorem with $n/2$ as approximation.

4.1-2 $k \times n \times n \times k$: each product is $k \times k$; there are n^2/k^2 such products? Actually: Multiply $(kn \times n)$ by $(n \times kn)$. The result is $kn \times kn$. Using recursive algorithm, each half-sized problem is $(kn/2 \times n/2) \times (n/2 \times kn/2)$. Running time $T(kn) = 8T(kn/2) + \Theta(1) \rightarrow \Theta((kn)^3) = \Theta(k^3 n^3)$. Similarly for $n \times kn \times kn \times n$: also $\Theta(k^3 n^3)$. Same asymptotic.

4.1-3 If copying instead of index calculation: each partition copies $\Theta(n^2)$ elements. Recurrence: $T(n) = 8T(n/2) + \Theta(n^2)$. Master theorem case 1: $n^{\log_8 8} = n^3$, $f(n) = n^2 = O(n^{3-\epsilon}) \rightarrow \Theta(n^3)$. Same solution.

4.1-4 Matrix addition:

```
MATRIX-ADD-RECURSIVE(A, B, C, n)
1 if n == 1
2    $c_{ij} = a_{ij} + b_{ij}$ 
3 else partition into  $n/2 \times n/2$  submatrices
4   MATRIX-ADD-RECURSIVE( $A_{ij}$ ,  $B_{ij}$ ,  $C_{ij}$ ,  $n/2$ )
5   MATRIX-ADD-RECURSIVE( $A_{ij}$ ,  $B_{ij}$ ,  $C_{ij}$ ,  $n/2$ )
6   MATRIX-ADD-RECURSIVE( $A_{ij}$ ,  $B_{ij}$ ,  $C_{ij}$ ,  $n/2$ )
7   MATRIX-ADD-RECURSIVE( $A_{ij}$ ,  $B_{ij}$ ,  $C_{ij}$ ,  $n/2$ )
```

Recurrence: $T(n) = 4T(n/2) + \Theta(1) \rightarrow$ master case 1: $n^{\log_4 4} = n^2$, $\Theta(1) = O(n^{2-\epsilon}) \rightarrow \Theta(n^2)$. With $\Theta(n^2)$ copying: $T(n) = 4T(n/2) + \Theta(n^2) \rightarrow$ master case 2: $f(n) = \Theta(n^2) = \Theta(n^{\log_4 4}) \rightarrow \Theta(n^2 \lg n)$.

4.2-1 Strassen: compute product of 2×2 matrices. Show $S_1, \dots, S_7$, then $P_1, \dots, P_7$, then combine.

4.2-2 Pseudocode for Strassen: Follow the 4-step outline.

4.2-3 Largest k for 3×3 multiplication: Need a such that $T(n) = aT(n/3) + \Theta(n^2)$. Strassen gives $n^{\lg 7} \approx n^{2.81}$. We need $n^{\log_3 a} < n^{\lg 7}$. So $\log_3 a < \lg 7 \rightarrow a < 3^{\lg 7} = 7^{\lg 3} \approx 7^{1.585} \approx 22.85$. So $a \leq 22$. Running time: $\Theta(n^{\log_3 a})$. Since $a > 3^2 = 9$ (or thereabouts), we get various exponents.

4.2-4 Pan's methods: 68×68 with 132464 mults $\rightarrow$ exponent = $\log_{68}(132464) \approx 2.795$. 70×70 with 143640 $\rightarrow \log_{70}(143640) \approx 2.794$. 72×72 with 155424 $\rightarrow \log_{72}(155424) \approx 2.794$. All slightly better than Strassen's 2.807.

4.2-5 Complex multiplication: compute $P_0 = (a+b)(c+d) = ac+ad+bc+bd$, $P_1 = ac$, $P_2 = bd$. Then $ac-bd = P_1 - P_2$, $ad+bc = P_0 - P_1 - P_2$. Only 3 real multiplications.

4.2-6 Squaring to multiplication: Use identity $(A+B)^2 = A^2 + AB + BA + B^2$. Compute $(A+B)^2$, A^2 , B^2 . Then $AB = ((A+B)^2 - A^2 - B^2)/2$. Since squaring is $\Theta(n^\alpha)$, multiplication is also $\Theta(n^\alpha)$.

4.3-1 a. $T(n) = T(n-1) + n$: Guess $T(n) = O(n^2)$. Inductive: $T(n) \leq c(n-1)^2 + n = cn^2 - 2cn + c + n \leq cn^2$ for $c \geq 1$, n large enough. b. $T(n) = T(n/2) + \Theta(1)$: Guess $O(\lg n)$. $T(n) \leq c \lg(n/2) + d = c \lg n - c + d \leq c \lg n$ for $c \geq d$. c. $T(n) = 2T(n/2) + n$: Guess $\Theta(n \lg n)$. Upper bound: $T(n) \leq 2c(n/2)\lg(n/2) + n = cn \lg n - cn + n \leq cn \lg n$ for $c \geq 1$. Lower bound: similarly. d. $T(n) = 2T(n/2+17)+n$: Guess $O(n \lg n)$. When n large, $n/2+17$ is close to $n/2$. $T(n) \leq 2c(n/2+17)\lg(n/2+17)+n \leq 2c \cdot (n/2+17)\lg(3n/4)+n$ for large n . Can show $\leq cn \lg n$ for appropriate c . e. $T(n) = 2T(n/3) + \Theta(n)$: Master case 3: $n^{\log_2 2} \approx n^{0.63}$, $f(n)=n = \Omega(n^{0.63+\epsilon})$. Regularity: $2 \cdot (n/3) = 2n/3 \leq c \cdot n$ for $c=3/4$. So $\Theta(n)$. f. $T(n) = 4T(n/2) + \Theta(n)$: Master case 1: $n^{\log_2 4} = n^2$, $f(n)=n = O(n^{2-\epsilon})$. So $\Theta(n^2)$.

4.3-2 $T(n) = 4T(n/2) + n$. Guess $\Theta(n^2)$. Substitution $T(n) \leq cn^2$: $T(n) \leq 4c(n/2)^2 + n = cn^2 + n$, which doesn't imply $\leq cn^2$. Subtracting lower-order term: $T(n) \leq cn^2 - dn$. Then $T(n) \leq 4(c(n/2)^2 - d(n/2)) + n = cn^2 - 2dn + n = cn^2 - dn - (dn - n) \leq cn^2 - dn$ for $d \geq 1$.

4.3-3 $T(n) = 2T(n-1) + 1$. Guess $O(2^n)$. $T(n) \leq 2 \cdot c \cdot 2^{n-1} + 1 = c \cdot 2^n + 1$, doesn't close. Subtract: $T(n) \leq c \cdot 2^n - d$. Then $T(n) \leq 2(c \cdot 2^{n-1} - d) + 1 = c \cdot 2^n - 2d + 1 \leq c \cdot 2^n - d$ when $d \geq 1$.

4.4-1 a. $T(n) = T(n/2) + n^3$: Tree height $\lg n$. Root cost n^3 , then $(n/2)^3$, $(n/4)^3 \dots$ Sum $= n^3(1 + 1/8 + 1/64 + \dots) = \Theta(n^3)$. Verify substitution: $T(n) \leq c(n/2)^3 + n^3 = (c/8+1)n^3 \leq cn^3$ for $c \geq 8/7$. b. $T(n) = 4T(n/3) + n$: Watershed $n^{\log_3 4} \approx n^{1.26}$. Case 1: $n = O(n^{1.26-\epsilon}) \rightarrow \Theta(n^{\log_3 4})$. c. $T(n) = 4T(n/2) + n$: Watershed $n^{\log_2 4} = n^2$. Case 1: $n = O(n^{2-\epsilon}) \rightarrow \Theta(n^2)$. d. $T(n) = 3T(n-1) + 1$: Tree height n , each level has 3^x nodes, leaf level has 3^{n-1} nodes, total $= \Theta(3^n)$.

4.4-2 $L(n) = L(n/3) + L(2n/3)$, base $L(n)=1$ for $n < n_0$. Lower bound: $L(n) \geq c \cdot n$. $L(n) \geq c \cdot n/3 + c \cdot 2n/3 = c \cdot n$. Base: $c=1$ works. So $L(n) = \Omega(n)$. With upper bound $L(n) = O(n)$ shown in text, $L(n) = \Theta(n)$.

4.4-3 $T(n) = T(n/3) + T(2n/3) + cn$. Prove $T(n) = \Omega(n \lg n)$. $T(n) \geq c \cdot n + c \cdot n/3 \cdot \lg(n/3) + c \cdot 2n/3 \cdot \lg(2n/3) = \dots \geq d \cdot n \lg n$ for some d . Combined with upper bound $O(n \lg n) \rightarrow \Theta(n \lg n)$.

4.4-4 $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n)$. By symmetry, guess $\Theta(n \lg n)$. Recursion tree has longest path from root through $(1-\alpha)$ splits.

4.5-1 a. $T(n) = 2T(n/4) + 1$: $n^{\lceil \log_2 n \rceil} = n^{\lceil 0.5 \rceil}$. Case 1: $1 = O(n^{\lceil 0.5 - \epsilon \rceil}) \rightarrow \Theta(\sqrt{n})$. b. $T(n) = 2T(n/4) + \sqrt{n}$: $n^{\lceil 0.5 \rceil}$. Case 2 ($k=0$): $\Theta(\sqrt{n} \lg n)$. c. $T(n) = 2T(n/4) + n^{\lceil 0.51 \rceil}$: $n^{\lceil 0.5 \rceil}$, $f(n) = n^{\lceil 0.51 \rceil} = \Omega(n^{\lceil 0.5 + \epsilon \rceil})$. Regularity: $2 \cdot (n/4)^{\lceil 0.51 \rceil} = 2 \cdot n^{\lceil 0.51 \rceil/4} \lceil 0.51 \rceil = 2 \cdot n^{\lceil 0.51 \rceil/2} \lceil 1.02 \rceil = n^{\lceil 0.51 \rceil/2} \lceil 0.02 \rceil \approx 0.986 \cdot n^{\lceil 0.51 \rceil} \leq c \cdot n^{\lceil 0.51 \rceil}$ for $c = 0.986 < 1$. Case 3: $\Theta(n^{\lceil 0.51 \rceil})$. d. $T(n) = 2T(n/4) + n$: $n^{\lceil 0.5 \rceil}$. $f(n) = n = \Omega(n^{\lceil 0.5 + \epsilon \rceil})$. Regularity: $2 \cdot (n/4) = n/2 \leq c \cdot n$ for $c = 1/2$. Case 3: $\Theta(n)$. e. $T(n) = 2T(n/4) + n^2$: $n^{\lceil 0.5 \rceil}$. $f(n) = n^2 = \Omega(n^{\lceil 0.5 + \epsilon \rceil})$. Regularity: $2 \cdot (n/4)^2 = n^2/8 \leq c \cdot n^2$ for $c = 1/8$. Case 3: $\Theta(n^2)$.

4.5-2 Professor Caesar: each matrix divided into $n/4 \times n/4$ submatrices. Watershed: $n^{\lceil \log_2 a \rceil}$. Strassen: $n^{\lceil \lg 7 \rceil} \approx n^{\lceil 2.807 \rceil}$. Need $n^{\lceil \log_2 a \rceil} < n^{\lceil \lg 7 \rceil} \rightarrow \log_2 a < \lg 7 \rightarrow a < 4^{\lceil \lg 7 \rceil} = 7^{\lceil \lg 4 \rceil} = 7^2 = 49$. So largest integer $a \leq 48$. (Since $\log_2 49 \approx 2.807$.)

4.5-3 Binary search: $T(n) = T(n/2) + \Theta(1)$. Watershed: $n^{\lceil \log_2 1 \rceil} = n^0 = 1$. Case 2 ($k=0$): $\Theta(\lg n)$.

4.5-4 $f(n) = \lg n$. Regularity: $af(n/b) \leq cf(n)$ with $a=1$, $b=2$: $\lg(n/2) \leq c \lg n \rightarrow \lg n - 1 \leq c \lg n \rightarrow 1 - 1/\lg n \leq c$. For n large, $1 - 1/\lg n \rightarrow 1$, so no $c < 1$ works. For case 3 condition: $f(n) = \Omega(n^{\lceil \log_b a + \epsilon \rceil}) = \Omega(n^{\lceil \epsilon \rceil})$. But $\lg n \neq \Omega(n^{\lceil \epsilon \rceil})$ for any $\epsilon > 0$.

4.5-5 $f(n) = 2^{\lceil \lg n \rceil}$. For any n , $f(n) \approx n$ (varies by factor 2). Show fails regularity: $af(n/b) \leq cf(n)$. With a, b chosen: $2^{\lceil \lg n/b \rceil} \leq c \cdot 2^{\lceil \lg n \rceil}$. The jumps at powers of 2 violate this.

4.6-1 Show $\sum_{j=0}^{\lceil \log_b n \rceil} a^j \lg^{k(n/b^j)} = \Theta(n^{\lceil \log_b a \rceil} \lg^{k+1} n)$.

4.6-2 Case 3 condition $f(n) = \Omega(n^{\lceil \log_b a + \epsilon \rceil})$ is implied by the regularity condition. Proof: If $af(n/b) \leq cf(n)$, then $f(n) \geq (a/c)f(n/b) \geq \dots \geq (a/c)^j f(n/b^j)$. Setting $n/b^j = n^0$ gives $f(n) \geq (a/c)^{\lceil \log_b(n/n^0) \rceil} \cdot \Theta(1) = \Theta(n^{\lceil \log_b(a/c) \rceil}) = \Omega(n^{\lceil \log_b a + \delta \rceil})$ where $\delta = -\log_b c > 0$ since $c < 1$.

4.6-3 For $f(n) = n/\lg n$ with $a=2, b=2$: Show $\sum a^j f(n/b^j) = \Theta(n \lg \lg n)$. Then $T(n) = \Theta(n) + \Theta(n \lg \lg n) = \Theta(n \lg \lg n)$.

4.7-1 If T satisfies $T(n) = \sum a_i T(n/b_i) + f(n)$, then $T'(n) = cT(n)$ satisfies $T'(n) = \sum a_i T'(n/b_i) + c \cdot f(n)$. So any constant factor on f can be absorbed by scaling initial conditions.

4.7-2 $f(n) = n^2$ satisfies polynomial-growth: for any $\psi \geq 1$, $f(\psi n)/f(n) = \psi^2$, and $1 \leq \psi \leq \psi^2$ means $1 \leq \psi^2 \leq \psi^2$, so d can be ψ^2 . $f(n) = 2^n$ fails: $f(\psi n)/f(n) = 2^{(\psi-1)n}$ which is unbounded for fixed $\psi > 1$ as $n \rightarrow \infty$.

4.7-3 f satisfies polynomial-growth: for any $\psi \geq 1$, $f(\psi n) \geq f(n)/d$. Take $\psi=1$, $\psi=1$, we get $f(n) \geq f(n)/d$. For large n , $f(n)$ positive since $d>1$ and $f(n)$ nonnegative.

4.7-4 Example: $f(n) = n^{1+\sin n}$. Then $f(\theta(n))$ can be $\theta(f(n))$ because the oscillation is scale-invariant, but polynomial-growth fails because near $\sin n = -1$, $f(\psi n)$ can be much larger relative to $f(n)$.

4.7-5 Akra-Bazzi solutions: a. $T(n)=T(n/2)+T(n/3)+T(n/6)+n \lg n$: $(1/2)^p+(1/3)^p+(1/6)^p=1 \rightarrow p=1$. $\int_1^n (u \lg u)/u^2 du = \theta(\lg^2 n)$. $T(n)=\theta(n) + \theta(n \lg^2 n)=\theta(n \lg^2 n)$. b. $T(n)=3T(n/3)+8T(n/4)+n^2/\lg n$: $3 \cdot (1/3)^p+8 \cdot (1/4)^p=1 \rightarrow p \approx 1$. etc. c-g: Similar approach.

Ch. 5 – Probabilistic Analysis and Randomized Algorithms

Named Entities (Terms & Definitions)

- **probabilistic analysis**: Using probability theory to analyze problems/algorithms, computing expected behavior under assumed input distribution.
- **average-case running time**: Expected running time over a distribution of inputs.
- **randomized algorithm**: Algorithm whose behavior depends on both input and values produced by a random-number generator.
- **expected running time**: Running time expectation over the algorithm's own random choices (contrast with average-case).
- **indicator random variable**: $I\{A\} = 1$ if event A occurs, 0 otherwise. $E[I\{A\}] = \Pr\{A\}$.
- **uniform random permutation**: A permutation of n elements where each of the $n!$ permutations is equally likely.
- **random-number generator $\text{RANDOM}(a,b)$** : Returns integer between a and b inclusive, each equally likely, independent of previous calls.
- **birthday paradox**: With only 23 people in a room, probability $\geq 1/2$ that two share a birthday.
- **coupon collector's problem**: Expected number of randomly obtained coupons to collect all b types is $b \cdot \ln b + O(b)$.
- **k-permutation**: A sequence containing k of n elements without repetitions.
- **inversion**: Pair (i,j) where $i < j$ and $A[i] > A[j]$.
- **hat-check problem**: Expected number of customers getting own hat back when hats returned randomly = 1.

Processes / Algorithms / Pathways

HIRE-ASSISTANT (n)

HIRE-ASSISTANT(n)

```
1 best = 0    // dummy least-qualified candidate
2 for i = 1 to n
3     interview candidate i
4     if candidate i is better than candidate best
5         best = i
6         hire candidate i
```

- **Cost:** Interview cost = $O(c_i \cdot n)$. Hiring cost = $O(c_h \cdot m)$ where m = number hired.
- **Worst-case:** Candidates in increasing order $\rightarrow$ hire all $n \rightarrow O(c_h \cdot n)$.
- **Average-case** (random order): Expected hires = $H_n = \ln n + \theta(1) \rightarrow O(c_h \cdot \ln n)$.

RANDOMIZED-HIRE-ASSISTANT (n)

RANDOMIZED-HIRE-ASSISTANT(n)

```
1 randomly permute the list of candidates
2 HIRE-ASSISTANT(n)
```

- **Expected hiring cost:** $O(c_h \cdot \ln n)$ for ANY input.

RANDOMLY-PERMUTE (A, n)

RANDOMLY-PERMUTE(A, n)

```
1 for i = 1 to n
2     swap A[i] with A[RANDOM(i, n)]
```

- **Goal:** Generate uniform random permutation.
- **Complexity:** $\theta(n)$, in-place.
- **Correctness:** Loop invariant – after $i-1$ iterations, each $(i-1)$ -permutation appears with probability $(n-i+1)!/n!$. At end ($i=n+1$), each n -permutation has probability $1/n!$.

ONLINE-MAXIMUM (k, n)

ONLINE-MAXIMUM(k, n)

```
1 best-score =  $-\infty$ 
2 for i = 1 to k
3     if score(i) > best-score
4         best-score = score(i)
5 for i = k+1 to n
6     if score(i) > best-score
```

```

7         return i
8 return n

```

- **Strategy:** Reject first k , then hire first candidate better than all seen.
- **Success probability:** $\Pr\{S\} = (k/n)(\ln n - \ln k) \pm O(1/n)$
- **Optimal k :** $k = n/e \approx 0.368n$, success probability $\geq 1/e \approx 0.368$.

RANDOM-SAMPLE (m, n)

RANDOM-SAMPLE(m, n)

```

1 S = {}
2 for k = n-m+1 to n
3     i = RANDOM(1, k)
4     if i < S
5         S = S ∪ {k}
6     else S = S ∪ {i}
7 return S

```

- **Goal:** Generate random m -subset of $\{1, \dots, n\}$ using only m calls to RANDOM.

Formulas & Equations

Indicator random variable

$I\{A\} = \{ 1 \text{ if } A \text{ occurs; } 0 \text{ otherwise } \}$ $E[I\{A\}] = \Pr\{A\}$ (Lemma 5.1)

Linearity of expectation

$E[\sum X_i] = \sum E[X_i]$ (holds even for dependent variables)

Expected hires in HIRE-ASSISTANT

- $\Pr\{\text{candidate } i \text{ is hired}\} = 1/i$
- $E[\text{number of hires}] = \sum_{i=1}^n 1/i = H_n = \ln n + \gamma + O(1/n) \approx \ln n$

Birthday paradox

- $\Pr\{k \text{ people all have different birthdays}\} = \prod_{i=1}^{k-1} (1 - i/n)$
- Using $1+x \leq e^x$: $\Pr\{\text{different}\} \leq e^{-k(k-1)/(2n)}$
- $\Pr\{\text{same}\} \geq 1/2$ when $k \geq (1+\sqrt{1+8n \ln 2})/2$. For $n=365$: $k \approx 23$.
- Expected number of matching pairs: $C(k,2)/n \approx k^2/(2n)$. For $k=28$, ≈ 1.04 .

Balls and bins (coupon collector)

- Expected tosses until every bin has a ball: $b \cdot H_b = b \ln b + O(b)$
- Per-stage expectation: $E[n_i] = b/(b-i+1)$
- Total: $\sum_{i=1}^b b/(b-i+1) = b \cdot H_b$

Streaks of consecutive heads

- Expected longest streak in n fair coin flips: $\Theta(\lg n)$
- Upper bound: $\Pr\{\text{streak} \geq 2 \lg n\} \leq 1/n$
- Lower bound: $\Pr\{\text{streak} \geq (\lg n)/2\} \geq 1 - O(1/n)$

Online hiring success probability

$\Pr\{S\} = (k/n)(H_{n-1} - H_{k-1}) \approx (k/n)(\ln n - \ln k)$ Optimal $k = n/e$, success probability $\approx 1/e$

Comparisons

	Probabilistic Analysis	Randomized Algorithm
Input assumption	Input distribution known	No assumption (imposes randomness)
Running time	Average-case over inputs	Expected over algorithm's random choices
Example	HIRE-ASSISTANT with random order	RANDOMIZED-HIRE-ASSISTANT
Guarantee	For typical inputs	For every input

Edge Cases / Traps

- **Pairwise independence insufficient** for birthday paradox analysis (Exercise 5.4-4) – need full independence for exact probability, but linearity of expectation with indicator variables requires only pairwise independence for expectation.
- **PERMUTE-WITH-ALL vs RANDOMLY-PERMUTE**: Swapping with any element (PERMUTE-WITH-ALL) does NOT produce uniform random permutation – it produces only n^n equally likely outcomes, and $n!$ does not divide n^n .
- **PERMUTE-WITHOUT-IDENTITY**: Professor Kelp's procedure ($\text{RANDOM}(i+1, n)$) never produces identity permutation but doesn't produce all other permutations uniformly.
- **PERMUTE-BY-CYCLE**: Each element has $1/n$ probability in any position, but permutations are not uniformly random (only n possible outcomes, not $n!$).
- **Average-case $\neq$ expected**: Average-case is over input distribution; expected is over algorithm's random choices.

- **Worst-case input for randomized algorithm:** No single input elicits worst-case behavior; unlucky random choices do.

End-of-Chapter Material

5.1-1 Being able to determine which candidate is best means you have a total order: for any two candidates, you can decide which is better. The ranking provides a total order since ranks are unique.

5.1-2 Implement `RANDOM(a,b)` using `RANDOM(0,1)`:

```

RANDOM(a,b)
1 range = b-a+1
2 bits = ⌈lg range⌉
3 do
4     result = 0
5     for i = 0 to bits-1
6         result = 2*result + RANDOM(0,1)
7 while result ≥ range
8 return a + result

```

Expected calls to `RANDOM(0,1)`: at most $2 \cdot \lceil \lg(b-a+1) \rceil$ (due to rejection sampling).

5.1-3 Unbiased from biased:

```

UNBIASED
1 do
2     x = BIASED-RANDOM()
3     y = BIASED-RANDOM()
4 while x == y
5 return x

```

Probability of $(0,1)$ followed by $(1,0) = p(1-p)$. Each equally likely. Expected iterations: $1/(2p(1-p))$.

5.2-1 Hire exactly once: first candidate is best $\rightarrow 1/n$. Hire exactly n times: candidates in increasing order $\rightarrow 1/n!$.

5.2-2 Hire exactly twice: first candidate is not best. Let best be at position $i > 1$. No one before best except first is hired. First must be best among first $i-1$, and second is overall best.
 $\Pr = \sum_{i=2}^n (1/(i-1)) \cdot (1/n) = (1/n) \cdot H_{n-1}$. Approximately $(\ln n)/n$.

5.2-3 Sum of n dice: $E[X] = \sum_{i=1}^n E[X_i] = n \cdot 3.5 = 3.5n$.

5.2-4 Independent dice: sum expectation = $3.5+3.5=7$. Second die = first: sum expectation = $3.5+3.5=7$. Second = 7 -first: $E[\text{sum}] = E[\text{first} + (7-\text{first})] = 7$. Linearity holds.

5.2-5 Hat-check problem: $X_i = I\{\text{person } i \text{ gets own hat}\}$.
 $\Pr\{X_i=1\}=1/n$. $E[X] = \sum_{i=1}^n 1/n = 1$.

5.2-6 Expected inversions: For each pair (i,j) , $\Pr\{A[i]>A[j]\} = 1/2$. $E[\text{\#inversions}] = C(n,2) \cdot 1/2 = n(n-1)/4$.

5.3-1 Rewrite RANDOMLY-PERMUTE to start with $i=0$ and treat $A[1:0]$ as nonempty by starting with $i=1$... Could initialize with $A[1]$ fixed and loop from $i=2$.

5.3-2 PERMUTE-WITHOUT-IDENTITY: Fails – for $n=3$, produces permutations $(2,1,3)$, $(3,1,2)$, $(2,3,1)$, $(3,2,1)$ each with probability $1/4$, never identity $(1,2,3)$ nor $(1,3,2)$. Not uniform over non-identity.

5.3-3 PERMUTE-WITH-ALL: Swap each $A[i]$ with $A[\text{RANDOM}(1,n)]$. Produces n^n possible sequences, each equally likely. Since $n!$ does not divide n^n in general (e.g., $n=3$: $3^n=27$, $3!=6$, $27/6$ not integer), permutations cannot be equally likely.

5.3-4 PERMUTE-BY-CYCLE: Each element has $1/n$ chance for any position (due to offset uniform from $1..n$). But only n possible permutations (one per offset value). Not uniform unless $n=1$.

5.3-5 RANDOM-SAMPLE: Proven by induction that each m -subset is equally likely. At step k , element i is added to S with prob i/k or something... Produces uniform random m -subset in m calls.

5.4-1 Someone has same birthday as you: $\Pr\{\text{someone matches}\} = 1 - (364/365)^{k-1}$. Need k such that $1 - (364/365)^{k-1} \geq 1/2 \rightarrow k \geq 1 + \ln(1/2)/\ln(364/365) \approx 254$. (Approximately 253.) Two people born on July 4: Use Poisson approx. Expected number $= C(k,2)/365$. Need $\approx 1 \rightarrow k \approx \sqrt{730} \approx 27$.

5.4-2 Need k such that $\Pr\{\text{different}\} \leq 0.01$. $e^{-k(k-1)/(2 \cdot 365)} \leq 0.01 \rightarrow k(k-1) \geq 2 \cdot 365 \cdot \ln 100 \approx 3361 \rightarrow k \geq 58$. (Actually ~ 57 .) Expected pairs $= C(57,2)/365 \approx 4.37$.

5.4-3 Birthday paradox variant: Expected tosses until bin has 2 balls $\approx \sqrt{(\pi b/2)}$. This is the expected number of tosses until a collision in hashing. More precisely, $E[T] \approx \sqrt{(\pi b/2)}$ where T is the first time a bin gets a second ball.

5.4-4 Mutual independence NOT required for the indicator variable method (only linearity of expectation which needs no independence). For exact probability calculation, pairwise independence is not sufficient for the product formula (need full independence). But indicator approach using linearity of expectation only needs pairwise expectations, which only need pairwise independence or even just marginal probabilities.

5.4-5 Three people with same birthday: Use Poisson. Expected number of triples $= C(k,3)/365^2$. Need $\approx 1 \rightarrow k^3 \approx 6 \cdot 365^2 \approx 799,350 \rightarrow k \approx 93$.

5.4-6 Probability a k-string over alphabet size n forms a k-permutation: $P = n!/((n-k)! \cdot n^k)$. This is exactly the birthday paradox event: all k items distinct.

5.4-7 n balls into n bins: - Expected empty bins: $E[\#empty] = \sum \Pr\{\text{bin } i \text{ empty}\} = n \cdot (1-1/n)^n \rightarrow n \cdot (1/e) = n/e$. - Expected bins with exactly one ball: $n \cdot C(n,1) \cdot (1/n) \cdot (1-1/n)^{n-1} = n \cdot (1-1/n)^{n-1} \rightarrow n/e$.

5.4-8 Show $\Pr\{\text{streak} \geq \lg n - 2 \lg \lg n\} \geq 1 - 1/n$. Partition into $n/(\lg n - 2 \lg \lg n)$ groups. Each group all-heads prob = $(1/2)^s$. Expected number of such streaks $\approx n/2^s = n/2^{\lg n - 2 \lg \lg n} = n/(n/(\lg n)^2) = (\lg n)^2$.

End of Chapters 1-5 Study Guide

Ch. 6 – Heapsort

Named Entities (Terms & Definitions)

- **Heap (Binary)**: Array object viewed as nearly complete binary tree; completely filled on all levels except possibly the lowest, which fills from left.
- **Max-heap**: For every node $i \neq \text{root}$: $A[\text{PARENT}(i)] \geq A[i]$ – largest element at root.
- **Min-heap**: For every node $i \neq \text{root}$: $A[\text{PARENT}(i)] \leq A[i]$ – smallest element at root.
- **Heap property**: The condition (max or min) that values in nodes satisfy.
- **Height of a node**: Number of edges on longest simple downward path to a leaf.
- **Height of a heap**: Height of its root = $\Theta(\lg n)$.
- **Priority queue**: Data structure maintaining a set S of elements, each with an associated key.
- **Max-priority queue**: Supports INSERT, MAXIMUM, EXTRACT-MAX, INCREASE-KEY.
- **Min-priority queue**: Supports INSERT, MINIMUM, EXTRACT-MIN, DECREASE-KEY.
- **Handle**: Additional info stored to map between application objects and array indices.

Processes / Algorithms / Pathways

PARENT / LEFT / RIGHT

```
PARENT(i) = ⌊ i/2 ⌋    // shift right 1 bit
LEFT(i)  = 2i         // shift left 1 bit
RIGHT(i) = 2i + 1     // shift left 1 bit + 1
```

MAX-HEAPIFY(A, i)

- **Goal:** Correct a single violation at node i , assuming children are max-heaps.
- **Input:** Array A with heap-size, index i .
- **Output:** Subtree at i obeys max-heap property.
- **Steps:** (1) Find largest among $A[i]$, $A[\text{LEFT}(i)]$, $A[\text{RIGHT}(i)]$; (2) If $A[i]$ not largest, swap $A[i]$ with $A[\text{largest}]$; (3) Recursively MAX-HEAPIFY on largest.
- **Complexity:** $O(\lg n) = O(h)$. Recurrence: $T(n) \leq T(2n/3) + \Theta(1) \rightarrow O(\lg n)$.
- **Edge Cases:** Leaf ($i > A.\text{heap-size}/2$) $\rightarrow$ no effect; already largest $\rightarrow$ no swaps.

BUILD-MAX-HEAP(A, n)

BUILD-MAX-HEAP(A, n)

```
1 A.heap-size = n
2 for i =  $\lfloor n/2 \rfloor$  downto 1
3     MAX-HEAPIFY(A, i)
```

- **Loop invariant:** Nodes $i+1, i+2, \dots, n$ are roots of max-heaps.
- **Complexity:** $O(n)$. Tight bound: $\sum_{h=0}^{\lfloor \lg n \rfloor} \lfloor n/2^{h+1} \rfloor \cdot O(h) = O(n)$.
- **Correctness:** Leaves (indices $\lfloor n/2 \rfloor + 1..n$) are trivial max-heaps; process bottom-up.

HEAPSORT(A, n)

HEAPSORT(A, n)

```
1 BUILD-MAX-HEAP(A, n)
2 for i = n downto 2
3     exchange A[1] with A[i]
4     A.heap-size = A.heap-size - 1
5     MAX-HEAPIFY(A, 1)
```

- **Loop invariant:** $A[1:i]$ is max-heap of i smallest elements; $A[i+1:n]$ contains $n-i$ largest elements, sorted.
- **Complexity:** $O(n \lg n)$. BUILD-MAX-HEAP $O(n)$ + $(n-1) \cdot \text{MAX-HEAPIFY } O(\lg n)$ each.
- **In-place:** Yes. **Stable:** No.

Priority Queue Operations

- **MAXIMUM:** $O(1)$ – return $A[1]$.
- **EXTRACT-MAX:** $O(\lg n)$ – replace root with last, shrink heap, MAX-HEAPIFY.
- **INCREASE-KEY:** $O(\lg n)$ – set key, bubble up while parent is smaller.
- **INSERT:** $O(\lg n)$ – add leaf with $-\infty$, then INCREASE-KEY to target.

Data Structures & Types

- **Heap:** Nearly complete binary tree in array $A[1:n]$. Height = $\lceil \lg n \rceil$.
- **Operations table:**

Operation	Time
MAX-HEAPIFY	$O(\lg n)$
BUILD-MAX-HEAP	$O(n)$
HEAPSORT	$O(n \lg n)$
MAXIMUM	$O(1)$
EXTRACT-MAX	$O(\lg n)$
INCREASE-KEY	$O(\lg n)$
INSERT	$O(\lg n)$

Comparisons & Trade-offs

Property	Heapsort	Mergesort	Quicksort
Time	$O(n \lg n)$	$O(n \lg n)$	$\Theta(n^2)$ worst, $\Theta(n \lg n)$ avg
In-place	Yes	No	Yes
Stable	No	Yes	No
Constant factors	Larger than quicksort	Larger	Small

Formulas & Equations

- Parent: $\text{PARENT}(i) = \lfloor i/2 \rfloor$
- Left child: $\text{LEFT}(i) = 2i$
- Right child: $\text{RIGHT}(i) = 2i + 1$
- Leaves: indices $\lfloor n/2 \rfloor + 1$ through n
- Nodes at height h : $\leq \lfloor n/2^{h+1} \rfloor$
- Heap height: $\lceil \lg n \rceil$
- BUILD-MAX-HEAP cost: $\sum_{h=0}^{\lceil \lg n \rceil} \lfloor n/2^{h+1} \rfloor \cdot O(h) = O(n)$

Visual Patterns

- **Figure 6.1:** Max-heap as (a) binary tree with values, (b) array with parent-child lines.
- **Figure 6.2:** MAX-HEAPIFY($A, 2$) – violator floats down.
- **Figure 6.4:** HEAPSORT – tan = sorted largest values; blue = remaining heap.

Edge Cases & Common Pitfalls

- Empty heap: $A.\text{heap-size} = 0$.
- All elements equal: still $O(n \lg n)$ for heapsort.
- MAX-HEAPIFY on leaf: no effect.

- Sorted array as min-heap: ascending order satisfies min-heap property.

Proof & Argument Patterns

- **Loop invariant for BUILD-MAX-HEAP:** nodes $i+1..n$ are roots of max-heaps.
- **Loop invariant for HEAPSORT:** $A[1:i]$ is max-heap of i smallest; $A[i+1:n]$ has $n-i$ largest sorted.
- **MAX-HEAPIFY correctness:** structural induction on subtree height.

Ch. 7 – Quicksort

Named Entities (Terms & Definitions)

- **Pivot:** Element selected as the partition point.
- **Low side:** Elements $\leq$ pivot. **High side:** Elements $\geq$ pivot.
- **Hoare partition:** Original partition scheme; two pointers from ends.
- **Tail-recursion elimination:** Convert one recursive call to iterative to reduce stack depth.

Processes / Algorithms / Pathways

PARTITION(A, p, r)

```

PARTITION(A, p, r)
1 x = A[r]           // pivot
2 i = p - 1
3 for j = p to r-1
4     if A[j] ≤ x
5         i = i + 1
6         exchange A[i] with A[j]
7 exchange A[i+1] with A[r]
8 return i + 1

```

- **Loop invariant:** For any k : (1) $p \leq k \leq i \rightarrow A[k] \leq x$; (2) $i+1 \leq k \leq j-1 \rightarrow A[k] > x$; (3) $k = r \rightarrow A[k] = x$.
- **Complexity:** $\Theta(n)$ for subarray of size n .

QUICKSORT(A, p, r)

- **Steps:** (1) Partition around pivot; (2) Recursively sort left subarray; (3) Recursively sort right subarray.
- **Worst case:** $T(n) = T(n-1) + \Theta(n) \rightarrow \Theta(n^2)$ – happens when already sorted or reverse sorted.
- **Best case:** $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \lg n)$ – balanced partition.
- **Any constant split:** $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n) \rightarrow \Theta(n \lg n)$.
- **In-place:** Yes. **Stable:** No.

Randomized Quicksort

- RANDOMIZED-PARTITION picks random pivot; RANDOMIZED-QUICKSORT uses it.
- **Expected time:** $\Theta(n \lg n)$ on n distinct elements.
- **Key Lemma 7.2:** z_i compared with z_j ($i < j$) iff one is chosen as pivot before any other in $Z_{ij} = \{z_i, \dots, z_j\}$.
- **Key Lemma 7.3:** $\Pr\{z_i \text{ compared with } z_j\} = 2/(j-i+1)$.
- **Expected comparisons:** $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2/(j-i+1) < 2n \ln n = O(n \lg n)$.

Comparisons & Trade-offs

Algorithm	Best	Average	Worst	In-place	Stable
Quicksort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n^2)$	Yes	No
Mergesort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	No	Yes
Heapsort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	Yes	No
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	Yes	Yes

Edge Cases & Common Pitfalls

- **All elements equal:** PARTITION returns $r \rightarrow \Theta(n^2)$ worst-case.
- **Already sorted:** $\Theta(n^2)$ if always pick last element as pivot (Lomuto).
- **Killer adversary** (McIlroy): can force $\Theta(n^2)$ on virtually any implementation.

Dates & People

- C. A. R. Hoare: invented quicksort (1960/1962).
- N. Lomuto: Lomuto partition.
- McIlroy: "killer adversary" for quicksort.

Ch. 8 – Sorting in Linear Time

Named Entities (Terms & Definitions)

- **Comparison sort:** Determines order only by comparing elements. Lower bound: $\Omega(n \lg n)$.
- **Decision tree:** Full binary tree representing comparisons performed by a comparison sort.
- **Counting sort:** Sorts integers in range $0..k$ in $\Theta(n+k)$ time; not a comparison sort.
- **Stable sort:** Equal values appear in output in same order as input.
- **Radix sort:** Sorts digit by digit, LSB first, using stable sort.
- **Bucket sort:** Assumes uniform distribution over $[0,1)$; divides into buckets, sorts each.

Processes / Algorithms / Pathways

COUNTING-SORT(A, n, k)

```
COUNTING-SORT(A, n, k)
1 let B[1:n] and C[0:k] be new arrays
2 for i = 0 to k: C[i] = 0
3 for j = 1 to n: C[A[j]] = C[A[j]] + 1
4 // C[i] now = count of elements equal to i
5 for i = 1 to k: C[i] = C[i] + C[i-1]
6 // C[i] now = count of elements ≤ i
7 for j = n down to 1:
8     B[C[A[j]]] = A[j]
9     C[A[j]] = C[A[j]] - 1
10 return B
```

- **Complexity:** $\Theta(n + k) = \Theta(n)$ when $k = O(n)$.
- **Stable:** Yes (processing from n down to 1 preserves order).
- **Not in-place:** Uses two additional arrays.

RADIX-SORT(A, n, d)

```
RADIX-SORT(A, n, d)
1 for i = 1 to d
2     use stable sort to sort on digit i
```

- **Complexity:** $\Theta(d(n + k))$ where k = range of digits.
- **Optimal r** (bits per digit): if $b < \lceil \lg n \rceil$, choose $r = b \rightarrow \Theta(n)$; if $b \geq \lceil \lg n \rceil$, choose $r = \lceil \lg n \rceil \rightarrow \Theta(bn/\lg n)$.

BUCKET-SORT(A, n)

```
BUCKET-SORT(A, n)
1 let B[0:n-1] be new array of empty lists
2 for i = 1 to n: insert A[i] into B[ $\lfloor n \cdot A[i] \rfloor$ ]
3 for i = 0 to n-1: sort list B[i] with insertion sort
4 concatenate lists B[0], B[1], ..., B[n-1]
```

- **Assumption:** Input uniformly distributed over $[0,1)$.
- **Average case:** $\Theta(n)$. **Worst case:** $\Theta(n^2)$ (all in one bucket).

Rules, Laws & Theorems

Theorem 8.1 (Lower bound): Any comparison sort requires $\Omega(n \lg n)$ comparisons in worst case. - Proof: $n! \leq l \leq 2^h \rightarrow h \geq \lg(n!) = \Omega(n \lg n)$. **Corollary 8.2:** Heapsort and mergesort are asymptotically optimal comparison sorts.

Formulas & Equations

- Decision tree: $n! \leq l \leq 2^h \rightarrow h \geq \lg(n!) = \Omega(n \lg n)$
- Stirling: $\lg(n!) = n \lg n - n \lg e + \Theta(\lg n) \approx n \lg n - 1.44n$

- Counting sort: $\Theta(n + k)$
- Radix sort: $\Theta((b/r)(n + 2^r))$

Edge Cases & Common Pitfalls

- Bucket sort with skewed distribution: worst-case $\Theta(n^2)$.
- Counting sort with large k : $\Theta(n + k)$ becomes $\Theta(k)$ which may dominate.

Ch. 9 – Medians and Order Statistics

Named Entities

- **i -th order statistic**: i -th smallest element.
- **Median**: Lower median $\lfloor (n+1)/2 \rfloor$, upper median $\lceil (n+1)/2 \rceil$.
- **Selection problem**: Find i -th smallest from n distinct numbers.

Processes / Algorithms / Pathways

MINIMUM / MAXIMUM

- Minimum: $n-1$ comparisons (each non-winner loses at least once).
- Simultaneous min & max: $\leq 3 \lfloor n/2 \rfloor$ comparisons (pairwise method).

RANDOMIZED-SELECT(A, p, r, i)

RANDOMIZED-SELECT(A, p, r, i)

```

1 if p == r: return A[p]
2 q = RANDOMIZED-PARTITION(A, p, r)
3 k = q - p + 1
4 if i == k: return A[q]
5 elseif i < k: return RANDOMIZED-SELECT(A, p, q-1, i)
6 else: return RANDOMIZED-SELECT(A, q+1, r, i-k)
```

- **Key difference from quicksort**: Only recurses on one side.
- **Expected time**: $\Theta(n)$ (with probability $\geq 1/2$, pivot falls in middle half).
- **Worst case**: $\Theta(n^2)$ (unlucky partitions).

SELECT – Deterministic Linear-Time Selection

- Group $n/5$ groups of 5; find median of each group.
- Recursively find median of the $\lfloor n/5 \rfloor$ group medians $\rightarrow$ pivot.
- Partition around pivot; recurse on appropriate side.
- **Pivot guarantee**: Each side $\leq 7n/10$.
- **Recurrence**: $T(n) \leq T(n/5) + T(7n/10) + \Theta(n) \rightarrow T(n) = \Theta(n)$.
- **Theorem 9.3**: SELECT runs in $\Theta(n)$ worst-case time.

Comparisons & Trade-offs

Algorithm	Strategy	Worst-case	Expected Practical?	
MINIMUM	Sequential	$\Theta(n)$	—	Yes
MIN+MAX pair	Pairwise	$\lceil 3n/2 \rceil - 2$	—	Yes
RANDOMIZED-SELECT	Randomized D&C	$\Theta(n^2)$	$\Theta(n)$	Yes
SELECT (median-of-medians)	Deterministic D&C	$\Theta(n)$	$\Theta(n)$	No (high constant)

Edge Cases & Common Pitfalls

- n not divisible by 5: preprocessing loop handles it.
- All elements equal: works correctly despite analysis assumption.
- $i = 1$ (minimum): SELECT returns after preprocessing.

Ch. 10 – Elementary Data Structures

Named Entities (Terms & Definitions)

- **Stack**: LIFO (last-in, first-out). Operations: PUSH, POP, STACK-EMPTY.
- **Queue**: FIFO (first-in, first-out). Operations: ENQUEUE, DEQUEUE.
- **Doubly linked list**: Each node has key, prev, next pointers.
- **Singly linked list**: Only next pointer.
- **Circular list**: tail→head, head→tail.
- **Sentinel**: Dummy object L.nil representing NIL; simplifies boundary checks.
- **Row-major order**: $\text{index} = n(i-1) + j$ for 1-origin.

Processes / Algorithms / Pathways

- **PUSH(S, x)**: $O(1)$. Increment top, store x. Error on overflow.
- **POP(S)**: $O(1)$. Return top, decrement. Error on underflow.
- **ENQUEUE(Q, x)**: $O(1)$. Insert at tail, wrap around circularly.
- **DEQUEUE(Q)**: $O(1)$. Remove from head, wrap around.
- **LIST-SEARCH(L, k)**: $\Theta(n)$ worst. Linear scan.
- **LIST-PREPEND(L, x)**: $O(1)$. Insert at front.
- **LIST-DELETE(L, x)**: $O(1)$ given pointer to x (doubly linked).
- **Sentinel benefits**: Simplifies LIST-DELETE (no boundary checks); eliminates end-of-list check in search.

Comparisons & Trade-offs

Operation	Array	Doubly Linked List
Access k-th	$O(1)$	$\Theta(k)$

Operation Array Doubly Linked List

Insert/delete at front $\Theta(n)$ $O(1)$

Search $\Theta(n)$ $\Theta(n)$

Edge Cases & Common Pitfalls

- Stack/queue underflow/overflow.
- Empty list operations.
- Singly linked delete = $\Theta(n)$ (must find predecessor).
- Sentinel: never delete sentinel.

Ch. 11 – Hash Tables

Named Entities (Terms & Definitions)

- **Hash function** $h: U \rightarrow \{0, \dots, m-1\}$ maps keys to slots.
- **Collision**: Two keys hash to same slot.
- **Load factor**: $\alpha = n/m$ (average elements per slot).
- **Independent uniform hashing**: Each key's hash independent and uniform.
- **Chaining**: Each slot points to linked list of colliding keys.
- **Open addressing**: All elements stored in table itself; probes.
- **Universal hashing**: Family H is universal if $\Pr[h(k_1) = h(k_2)] \leq 1/m$ for distinct k_1, k_2 .
- **Perfect hashing**: Static keys, $O(1)$ worst-case search.

Processes / Algorithms / Pathways

Hashing with Chaining

- INSERT: $O(1)$ – prepend to chain.
- SEARCH: average $\Theta(1+\alpha)$ under independent uniform hashing.
- DELETE: $O(1)$ if doubly linked chain.
- **Theorem 11.1**: Unsuccessful search averages $\Theta(1+\alpha)$.
- **Theorem 11.2**: Successful search averages $\Theta(1+\alpha)$.

Open Addressing

HASH-INSERT(T, k)

```
1  $i = 0$ 
2 repeat
3    $q = h(k, i)$ 
4   if  $T[q] == \text{NIL}$ :  $T[q] = k$ ; return  $q$ 
5   else  $i = i + 1$ 
6 until  $i == m$ 
7 error "hash table overflow"
```

- $\alpha \leq 1$ (table can fill up).
- **Double hashing**: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Best for avoiding clustering.

- **Linear probing:** $h(k,i) = (h_0(k) + i) \bmod m$. Primary clustering; good cache locality.

Hashing Analysis

Operation Expected probes (open addressing)

Unsuccessful search $\leq 1/(1-\alpha)$

Successful search $\leq (1/\alpha) \ln(1/(1-\alpha))$

Universal Hash Family H_{pm}

- Choose prime $p > m$, $h_{\{ab\}}(k) = ((ak + b) \bmod p) \bmod m$.
- Theorem 11.4: H_{pm} is universal.

Edge Cases & Common Pitfalls

- All keys hash to same slot $\rightarrow O(n)$ search (chaining).
- Open addressing table full $\rightarrow$ overflow.
- Deletion in open addressing: need DELETED marker or special method.
- Double hashing with $\gcd(m, h_0(k)) > 1 \rightarrow$ only $1/d$ of table examined.

Dates & People

- H.P. Luhn (1953): hash tables with chaining.
- Carter & Wegman (1979): universal hashing.
- Thorup: 5-independence for linear probing analysis.

Ch. 12 – Binary Search Trees

Named Entities

- **BST property:** For node x , all keys in left subtree $\leq x.key \leq$ all keys in right subtree.
- **Inorder tree walk:** left $\rightarrow$ root $\rightarrow$ right $\rightarrow$ produces sorted order.

Processes / Algorithms / Pathways

TREE-SEARCH(x, k): $O(h)$. Recursive or iterative.

TREE-MINIMUM(x): $O(h)$. Follow left child until NIL.

TREE-MAXIMUM(x): $O(h)$. Follow right child until NIL.

TREE-SUCCESSOR(x): $O(h)$.

- If $x.right \neq \text{NIL}$: return TREE-MINIMUM($x.right$).

- Else: go up until finding node that is left child of its parent. ##### TREE-INSERT(T, z): $O(h)$. Trailing pointer y tracks parent. ##### TREE-DELETE(T, z): $O(h)$. Three cases:
 1. No left child: replace by right child.
 2. No right child but has left: replace by left child.
 3. Two children: find successor $y = \text{TREE-MINIMUM}(z.\text{right})$; replace z by y .

Formulas & Equations

- Inorder walk: $T(n) = T(k) + T(n-k-1) + d \rightarrow O(n)$ (Theorem 12.1).

Edge Cases & Common Pitfalls

- Linear chain: BST degenerates $\rightarrow O(n)$ for all operations.
- All keys equal: depends on tie-breaking strategy.
- Insert into empty tree: trailing pointer $y = \text{NIL} \rightarrow \text{set } T.\text{root} = z$.

Ch. 13 – Red-Black Trees

Named Entities (Terms & Definitions)

- **Red-black tree**: BST with one extra bit per node: color = RED or BLACK.
- **Black-height $bh(x)$** : Number of black nodes on path from x (excluding x) to leaf.
- **Sentinel $T.\text{nil}$** : All NIL pointers replaced by single black $T.\text{nil}$.

Properties

1. Every node is RED or BLACK.
2. Root is BLACK.
3. Every leaf ($T.\text{nil}$) is BLACK.
4. RED node's children are both BLACK (no two reds in a row).
5. Equal black count on all paths from node to leaves.

Lemma 13.1: RB-tree with n internal nodes has height $\leq 2 \lg(n+1)$.
 - Proof: Subtree at x has $\geq 2^{bh(x)} - 1$ nodes. $bh(\text{root}) \geq h/2 \rightarrow n \geq 2^{h/2} - 1$. - Corollary: SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR all $O(\lg n)$.

Processes / Algorithms / Pathways

LEFT-ROTATE(T, x): $O(1)$. Makes `x.right` become new root of subtree. Preserves BST property.

RB-INSERT(T, z): $O(\lg n)$

1. BST-insert `z`, color it RED.
2. RB-INSERT-FIXUP: three cases while `z.p` is RED:
 - **Case 1:** Uncle RED → recolor parent, uncle, grandparent; move `z` up 2.
 - **Case 2:** Uncle BLACK, `z` is right child → left-rotate parent → Case 3.
 - **Case 3:** Uncle BLACK, `z` is left child → recolor parent BLACK, grandparent RED; right-rotate grandparent.
 - Root always BLACK.

RB-DELETE(T, z): $O(\lg n)$

- Based on TREE-DELETE with RB-TRANSPLANT.
- Tracks `y` = node removed/moved and `y-original-color`.
- If `y` was BLACK: RB-DELETE-FIXUP(`x`) where `x` replaces `y`.
- **Four fixup cases** (`x` is left child):
 1. Sibling RED → recolor, left-rotate → becomes case 2/3/4.
 2. Sibling BLACK, both nephews BLACK → set sibling RED, move `x` up.
 3. Sibling BLACK, left nephew RED, right BLACK → recolor, right-rotate → case 4.
 4. Sibling BLACK, right nephew RED → recolor, left-rotate, done.

Formulas & Equations

- Height: $\leq 2 \lg(n+1)$.
- All operations: $O(\lg n)$.
- Rotations per INSERT: at most 2. Rotations per DELETE: at most 3.

Visual Patterns

- **Rotations** (Fig 13.2): `x` → `y` transform via constant pointer changes.
- **Insert cases** (Fig 13.4-13.6): case 1 = recolor+move up; case 2→3 via rotation.

Dates & People

- Bayer (1972): "symmetric binary B-trees".
- Guibas & Sedgwick: named red-black trees.
- AVL trees: Adel'son-Vel'skiĭ & Landis (1962).
- Splay trees: Sleator & Tarjan.

Ch. 14 – Dynamic Programming

Named Entities (Terms & Definitions)

- **Dynamic Programming:** Solves problems by combining solutions to overlapping subproblems. "Programming" refers to tabular method.
- **Optimal substructure:** Optimal solution contains within it optimal solutions to subproblems.
- **Overlapping subproblems:** Space of subproblems is small; same subproblems solved repeatedly.
- **Memoization:** Top-down recursive approach that saves results in a table.
- **Cut-and-paste technique:** Proof method: assume suboptimal subproblem, cut it out, paste optimal one → contradiction.
- **Independent subproblems:** Solution to one subproblem does not affect another.

Processes / Algorithms / Pathways

Four-Step DP Method

1. Characterize structure of optimal solution.
2. Recursively define value of optimal solution.
3. Compute value (typically bottom-up).
4. Construct optimal solution from computed info.

Rod Cutting

- **Problem:** Given rod length n and price table p_i , maximize revenue r_n by cutting rod.
- **Recurrence:** $r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$, $r_0 = 0$.
- **Naive CUT-ROD:** $T(n) = 2^n$ (exponential).
- **BOTTOM-UP-CUT-ROD:** $\Theta(n^2)$.
- **Number of ways to cut:** 2^{n-1} .

Matrix-Chain Multiplication

- **Problem:** Fully parenthesize chain $A_1 \dots A_n$ to minimize scalar multiplications.
- **Dimensions:** $p_{i-1} \times p_i$ for A_i .
- **Cost recurrence:** $m[i, i] = 0$; $m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} \cdot p_k \cdot p_j\}$.
- **MATRIX-CHAIN-ORDER:** $O(n^3)$ time, $\Theta(n^2)$ space.
- **Number of parenthesizations:** Catalan numbers $P(n) = \Omega(4^{n/3})$.

Longest Common Subsequence (LCS)

- **Problem:** Find maximum-length subsequence common to $X = \langle x_1 \dots x_m \rangle$, $Y = \langle y_1 \dots y_n \rangle$.

- **Recurrence:** $c[i,j] = 0$ if $i=0$ or $j=0$; $c[i-1,j-1] + 1$ if $x_i = y_j$; $\max(c[i-1,j], c[i,j-1])$ if $x_i \neq y_j$.
- **LCS-LENGTH:** $\Theta(mn)$ time, $\Theta(mn)$ space.

Optimal Binary Search Tree (OBST)

- **Problem:** BST minimizing expected search cost given p_i (key k_i) and q_i (dummy keys).
- **Cost recurrence:** $e[i,i-1] = q_{i-1}$; $e[i,j] = \min_{i \leq r \leq j} \{e[i,r-1] + e[r+1,j] + w(i,j)\}$.
- **OPTIMAL-BST:** $\Theta(n^3)$. Knuth optimization reduces to $\Theta(n^2)$.

Rules, Laws & Theorems

Theorem 14.1 (Optimal substructure of LCS): If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is LCS of X_{m-1} , Y_{n-1} . If $x_m \neq y_n$, then Z is LCS of X_{m-1} , Y or X , Y_{n-1} .

Comparisons & Trade-offs

Aspect	Top-Down (Memoization)	Bottom-Up
Control	Recursive	Iterative
Subproblems solved	Only required	All
Overhead	More	Less
Asymptotic	Same	Same

DP vs Divide-and-Conquer: DP has overlapping subproblems, D&C has disjoint.

DP vs Greedy: DP solves subproblems first then chooses; greedy chooses first then solves one subproblem.

Ch. 15 – Greedy Algorithms

Named Entities

- **Greedy algorithm:** Makes locally optimal choice hoping it leads to globally optimal solution.
- **Greedy-choice property:** Globally optimal solution can be assembled by making locally optimal choices.
- **Prefix-free code:** No codeword is prefix of any other.
- **Huffman code:** Optimal prefix-free code built by merging least-frequent characters.

Processes / Algorithms / Pathways

Activity Selection

- **Problem:** Select max-size set of mutually compatible activities (s_i start, f_i finish).

- **Greedy choice:** Pick activity with earliest finish time.
- **RECURSIVE-ACTIVITY-SELECTOR:** $\Theta(n)$ (sorted by finish).
- **GREEDY-ACTIVITY-SELECTOR:** $\Theta(n)$ iterative.

Huffman Codes

HUFFMAN(C)

```

1 Q = C
2 for i = 1 to |C|-1
3     allocate new node z
4     x = EXTRACT-MIN(Q); y = EXTRACT-MIN(Q)
5     z.left = x; z.right = y
6     z.freq = x.freq + y.freq
7     INSERT(Q, z)
8 return EXTRACT-MIN(Q)
```

- **Time:** $O(n \lg n)$ with binary min-heap.
- **Lemma 15.2:** Two lowest-frequency characters can be deepest siblings in optimal code.
- **Lemma 15.3:** Optimal substructure for Huffman codes.

Fractional vs 0-1 Knapsack

- **Fractional:** Greedy by value/weight works. $O(n \lg n)$.
- **0-1:** Greedy fails. DP $O(nW)$.

Offline Caching (Furthest-in-Future)

- Evict block whose next access is furthest in future.
- Theorem 15.5: Furthest-in-future optimal for caching with full future knowledge.

Rules, Laws & Theorems

Theorem 15.1 (Greedy choice for activity selection): Earliest-finish activity is included in some optimal solution.

Theorem 15.4: HUFFMAN produces optimal prefix-free code.

Comparisons & Trade-offs

Aspect	Greedy	DP
Approach	Top-down: choose first	Bottom-up: solve subproblems first
Subproblems	One remains	Many possible
Hallmarks	Greedy-choice + optimal substructure	Optimal substructure + overlapping
Example	Fractional knapsack	0-1 knapsack

Ch. 16 – Amortized Analysis

Named Entities

- **Amortized analysis:** Averages time over sequence of operations; guarantees average per-operation worst-case.
- **Aggregate analysis:** Total cost $T(n)$ / n = amortized cost per op.
- **Accounting method:** Overcharge some ops early, store credit, use for later undercharged ops.
- **Potential method:** Φ : data structure $\rightarrow \mathbb{R}$. $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.
- **Load factor:** $\alpha(T) = \text{num}/\text{size}$.

Processes / Algorithms / Pathways

MULTIPOP(S, k): $\min(s, k)$ actual cost.

Binary Counter INCREMENT: flips $t_i + 1$ bits (t_i = trailing 1s).

Dynamic Table: TABLE-INSERT expands by factor of 2 when full; TABLE-DELETE contracts at $\alpha < 1/4$.

Comparisons & Trade-offs

Method	Key concept	Amortized cost
Aggregate	Total cost / n	Same for all ops
Accounting	Credit on objects	Differs per type
Potential	$\Phi(D_i) - \Phi(D_{i-1})$	Differs per type

Formulas & Equations

- **Accounting constraint:** $\sum \hat{c}_i \geq \sum c_i$ (credit never negative).
- **Potential:** $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$; $\sum \hat{c}_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$.
- **Binary counter flips:** $\sum_{i=0}^{k-1} \lceil n/2^i \rceil < 2n$.
- **Dynamic table (insert only):** $\Phi = 2 \cdot \text{num} - \text{size}$. Amortized cost = 3 per insertion.
- **Dynamic table (insert+delete):** $\Phi = 2 \cdot \text{num} - \text{size}$ if $\alpha \geq 1/2$; $\Phi = \text{size}/2 - \text{num}$ if $\alpha < 1/2$.
- **TABLE-INSERT aggregate:** $\leq 3n$ total for n insertions.

Edge Cases & Common Pitfalls

- Empty stack MULTIPOP: while condition prevents errors.
- Counter not starting at 0: $\text{total} \leq 2n + b_0 - b_n$.
- Contraction thrashing: halving at $1/2$ causes $\Theta(n^2)$; solution: contract at $1/4$.

Ch. 17 – Augmenting Data Structures

Named Entities (Terms & Definitions)

- **Augment a data structure:** Add new fields and maintain invariants to support new operations.
- **Order-statistic tree:** Red-black tree augmented with $x.size = x.left.size + x.right.size + 1$.
- **Interval tree:** Red-black tree with intervals as keys, augmented with $x.max = \max(x.int.high, x.left.max, x.right.max)$.

Processes / Algorithms / Pathways

Dynamic Order Statistics

- **OS-SELECT(x, i):** $O(\lg n)$. Find i -th order statistic recursively by checking $x.left.size$.
- **OS-RANK(T, x):** $O(\lg n)$. Compute rank by walking up from x to root.
- **Maintaining size:** On INSERT, increment size along path ($O(\lg n)$). On rotation, update size of rotated nodes ($O(1)$).

Interval Trees

- **INTERVAL-SEARCH(T, i):** $O(\lg n)$. Find any interval in T that overlaps i .
- **Key:** At each node, go left if $x.left.max \geq i.low$; else go right.
- **Theorem 17.2:** INTERVAL-SEARCH returns an overlapping interval if one exists.

Rules, Laws & Theorems

Theorem 17.1 (Choosing augmentation): Can augment red-black trees with field f if: (1) f can be computed from node's own data, $left.f$, $right.f$ in $O(1)$; (2) f is updated during rotations in $O(1)$. Then all RB operations remain $O(\lg n)$.

Ch. 18 – B-Trees

Named Entities

- **B-tree:** Balanced search tree optimized for disk storage. Nodes have high branching factor.
- **Minimum degree t :** Each node (except root) has $t-1 \leq \text{keys} \leq 2t-1$. Internal nodes have degree $t..2t$.
- **B-tree height:** $h \leq \log_t ((n+1)/2)$ (Theorem 18.1).

Processes / Algorithms / Pathways

B-TREE-SEARCH(x, k): $O(t \log_t n)$. Linear search within node, then recurse.

B-TREE-SPLIT-CHILD(x, i): Splits full child y of x into two (median key moves up to x). $O(t)$.

B-TREE-INSERT(T, k): $O(t \log_t n)$. Split root if full; insert-nonfull recursively.

B-TREE-DELETE: Three main cases with subcases. $O(t \log_t n)$.

Visual Patterns

- B-tree of height 2 with minimum degree 3: internal nodes have 2-5 keys, children 3-6.

Comparisons & Trade-offs

- B-trees vs RB-trees: B-trees minimize disk accesses (high branching factor).
- B+ trees: all data in leaves, internal nodes just keys.
- B* trees: internal nodes at least $2/3$ full (not $1/2$).

Ch. 19 – Data Structures for Disjoint Sets

Named Entities

- **Disjoint-set data structure**: Maintains collection of disjoint dynamic sets.
- **Union by rank**: Root of shorter tree points to taller. Rank = upper bound on height.
- **Path compression**: During FIND-SET, make each node on path point directly to root.

Processes / Algorithms / Pathways

Operations

- **MAKE-SET**(x): $O(1)$. Create singleton set.
- **FIND-SET**(x): $O(\alpha(n))$ amortized. Follow parent pointers to root with path compression.
- **UNION**(x, y): $O(\alpha(n))$ amortized. FIND both roots; attach shorter to taller.

Linked-List Representation

- Each set: linked list with head/tail pointers; each node has pointer to set object.
- **Union naïve**: $O(n^2)$ total for n unions.

- **Weighted-union heuristic:** $O(m + n \lg n)$ for m ops, n of which are MAKE-SET.
- **Theorem 19.1:** Weighted-union gives $O(m + n \lg n)$ total time.

Disjoint-Set Forests

- Tree representation with union by rank + path compression.
- **Theorem 19.14:** $O(m \alpha(n))$ total time for m operations, where $\alpha(n)$ is inverse Ackermann function – grows extremely slowly ($\alpha(n) \leq 4$ for $n \leq 2^{65536}$).

Formulas & Equations

- $\alpha(n) = \min\{k : A_k(1) \geq n\}$ where $A_k(j)$ is Ackermann-like function.
- Rank properties: $\text{rank}(p[x]) \geq \text{rank}(x) + 1$; rank increases only on path compression.

Ch. 20 – Elementary Graph Algorithms

Named Entities (Terms & Definitions)

- **Graph $G = (V, E)$:** Vertices V and edges E .
- **Adjacency-list representation:** Array Adj of $|V|$ lists. Memory: $\Theta(V+E)$.
- **Adjacency-matrix representation:** $|V| \times |V|$ matrix. Memory: $\Theta(V^2)$.
- **Sparse graph:** $|E| \ll |V|^2$. **Dense:** $|E|$ close to $|V|^2$.
- **Breadth-first tree:** Predecessor subgraph with unique shortest path from s .
- **Depth-first forest:** DFS predecessor subgraph with timestamps.
- **Topological sort:** Linear ordering of DAG vertices where (u,v) implies u before v .
- **Strongly connected component (SCC):** Maximal $C \subseteq V$ where every pair mutually reachable.
- **Component graph G_{SCC} :** Contracted SCC graph – always a DAG.

Processes / Algorithms / Pathways

BFS(G, s): $O(V+E)$

- Queue of gray vertices. Discover vertices by distance.
- $v.d$ = shortest-path distance $\delta(s,v)$ (number of edges).
- Produces breadth-first tree.

DFS(G): $\Theta(V+E)$

- Discovery timestamps $u.d$, finish times $u.f$ ($1 \leq u.d < u.f \leq 2|V|$).
- Edge types: tree (white), back (gray), forward (black, descendant), cross (black, nondescendant).

- In undirected graphs: only tree and back edges.

TOPOLOGICAL-SORT(G): $\Theta(V+E)$

- Run DFS; output vertices in reverse order of finish times.
- Correct iff graph is acyclic.

STRONGLY-CONNECTED-COMPONENTS(G): $\Theta(V+E)$

1. DFS(G) to compute finish times.
2. Create G^T (transpose).
3. DFS(G^T) in order of decreasing u.f.
4. Output each DFS tree as SCC.

Rules, Laws & Theorems

- **Lemma 20.1:** $\delta(s,v) \leq \delta(s,u) + 1$ for any edge (u,v) .
- **Theorem 20.5:** BFS correctly computes shortest-path distances.
- **Theorem 20.7 (Parenthesis theorem):** DFS discovery/finish intervals are either disjoint or nested.
- **Theorem 20.9 (White-path theorem):** v is descendant of u iff at time $u.d$ there's white path from u to v .
- **Lemma 20.11:** Digraph is acyclic iff DFS has no back edges.
- **Theorem 20.16:** SCC algorithm correctly computes SCCs.

Visual Patterns

- BFS: waves emanating from source.
- DFS: depth-first forest with parentheses nesting structure.
- SCC: component graph G_{SCC} is acyclic; second DFS visits in topologically sorted order.

Ch. 21 – Minimum Spanning Trees

Named Entities

- **Spanning tree:** Acyclic subset $T \subseteq E$ connecting all vertices; $|V|-1$ edges.
- **Minimum spanning tree (MST):** Spanning tree with minimum total weight.
- **Cut $(S, V-S)$:** Partition of V .
- **Light edge:** Minimum-weight edge crossing a cut.
- **Safe edge:** Edge that can be added to A while $A \subseteq$ some MST.

Processes / Algorithms / Pathways

GENERIC-MST: Add safe edges greedily until spanning tree.

Kruskal's Algorithm: $O(E \lg V)$

- Sort edges by weight. Process smallest first. If edge connects different components (FIND-SET), add it (UNION).
- Uses disjoint-set data structure.
- Edge-based approach.

Prim's Algorithm: $O(E \lg V)$ with binary heap; $O(E + V \lg V)$ with Fibonacci heap

- Grow single tree from root. Use priority queue keyed by min edge weight to tree.
- Vertex-based approach.

Rules, Laws & Theorems

Theorem 21.1 (Cut property): If $A \subseteq$ some MST, cut $(S, V-S)$ respects A , (u,v) is light edge crossing cut $\rightarrow (u,v)$ is safe for A . **Cycle property:** Maximum-weight edge on any cycle is not in any MST.

Comparisons & Trade-offs

Algorithm	Data Structure	Time
Kruskal	Disjoint-set forest	$O(E \lg V)$
Prim (binary heap)	Binary min-heap	$O(E \lg V)$
Prim (Fibonacci heap)	Fibonacci heap	$O(E + V \lg V)$

Ch. 22 – Single-Source Shortest Paths

Named Entities

- **Shortest-path weight $\delta(u,v)$:** Min weight over all paths; ∞ if no path; $-\infty$ if negative cycle reachable.
- **Relaxation:** If $v.d > u.d + w(u,v)$, update $v.d = u.d + w(u,v)$, $v.\pi = u$.
- **Negative-weight cycle:** Makes shortest paths undefined for reachable vertices.

Processes / Algorithms / Pathways

Bellman-Ford Algorithm: $O(VE)$

- Initialize; relax all edges $|V|-1$ times; check for negative cycles.
- Returns FALSE if negative-weight cycle reachable.

- Works with negative weights.

DAG-Shortest-Paths: $\Theta(V+E)$

- Topological sort, then relax edges in topological order.
- Linear time! Works with negative weights. No cycles.

Dijkstra's Algorithm: $O(V^2)$ array, $O(E \lg V)$ binary heap, $O(V \lg V + E)$ Fibonacci heap

- Requires nonnegative edge weights.
- Greedy: always selects vertex with smallest estimate from $V-S$.

Rules, Laws & Theorems

- **Lemma 22.1 (Optimal substructure):** Subpaths of shortest paths are shortest paths.
- **Lemma 22.10 (Triangle inequality):** $\delta(s,v) \leq \delta(s,u) + w(u,v)$.
- **Lemma 22.11 (Upper-bound property):** $v.d \geq \delta(s,v)$ always; once equal, never changes.
- **Lemma 22.14 (Convergence property):** If $s \rightarrow \dots \rightarrow u \rightarrow v$ is shortest path and $u.d = \delta(s,u)$ before relaxing (u,v) , then $v.d = \delta(s,v)$ after.
- **Lemma 22.15 (Path-relaxation property):** If edges of shortest path relaxed in order, $v_k.d = \delta(s,v_k)$ afterward.

Comparisons & Trade-offs

Algorithm	Negative weights?	Time
BFS	Unweighted	$O(V+E)$
DAG-Shortest-Paths	Yes (no cycles)	$\Theta(V+E)$
Dijkstra	No	$O(V \lg V + E)$ (Fibonacci)
Bellman-Ford	Yes	$O(VE)$

Ch. 23 – All-Pairs Shortest Paths

Named Entities

- **Predecessor matrix Π :** π_{ij} = predecessor of j on shortest path from i .
- **$W = (w_{ij})$:** Input adjacency matrix.

Processes / Algorithms / Pathways

Floyd-Warshall Algorithm: $\Theta(V^3)$

- $D^{\{k\}}$ = shortest paths using intermediate vertices only from $\{1, \dots, k\}$.

- Recurrence: $d^{\{(k)\}}_{ij} = \min(d^{\{(k-1)\}}_{ij}, d^{\{(k-1)\}}_{ik} + d^{\{(k-1)\}}_{kj})$.
- Space-optimized: single matrix D overwritten in place.

Johnson's Algorithm: $O(V^2 \lg V + VE)$

- For sparse graphs. Reweight edges using Bellman-Ford ($h(v) = \delta(s, v)$).
- Run Dijkstra from each vertex.
- Reweighting: $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ preserves shortest paths.

Comparisons & Trade-offs

Algorithm	Time	Notes
Floyd-Warshall	$O(V^3)$	Simple, dense graphs
Johnson	$O(V^2 \lg V + VE)$	Best for sparse
Run Dijkstra	V	times

Ch. 24 – Maximum Flow

Named Entities

- **Flow network:** Directed graph (V, E) with source s , sink t , capacity $c(u, v) \geq 0$.
- **Flow f :** Capacity constraint ($0 \leq f(u, v) \leq c(u, v)$); flow conservation ($\forall u \neq s, t: \text{in} = \text{out}$).
- **Flow value $|f|$:** $\sum_v f(s, v) - \sum_v f(v, s)$.
- **Residual network G_f :** Edges with $c_f(u, v) > 0$.
- **Augmenting path:** $s \rightarrow t$ path in G_f .
- **Cut (S, T) :** Partition with $s \in S, t \in T$. Capacity = $\sum_{u \in S} \sum_{v \in T} c(u, v)$.

Processes / Algorithms / Pathways

Ford-Fulkerson Method

- While augmenting path exists in G_f , augment flow by min residual capacity on path.
- **Time:** $O(E \cdot |f^*|)$ with integer capacities. May not terminate with irrational capacities.

Edmonds-Karp: $O(VE^2)$

- Ford-Fulkerson where augmenting path found via BFS.
- Each edge becomes critical $\leq |V|/2$ times $\rightarrow O(VE)$ augmentations $\times O(E)$ per BFS.

Maximum Bipartite Matching via Flow

- Construct flow network: $s \rightarrow L$ (cap 1), $L \rightarrow R$ (cap 1), $R \rightarrow t$ (cap 1).
- Max flow = max matching cardinality.
- **Integrality theorem:** Ford-Fulkerson with integer capacities produces integer flow.

Rules, Laws & Theorems

Theorem 24.6 (Max-flow min-cut theorem): TFAE: (1) f is max flow; (2) G_f has no augmenting path; (3) $|f| = c(S, T)$ for some cut (S, T) . **Lemma 24.4:** Net flow across any cut equals $|f|$. **Corollary 24.5:** $|f| \leq c(S, T)$ for any cut.

Ch. 25 – Matchings in Bipartite Graphs

Named Entities

- **Matching:** Subset $M \subseteq E$ with at most one incident edge per vertex.
- **Perfect matching:** Every vertex matched.
- **M-alternating path:** Edges alternate between M and $E-M$.
- **M-augmenting path:** Alternating path with both endpoints unmatched.
- **Stable matching:** No blocking pair (two people who prefer each other to current partners).
- **Assignment problem:** Find max-weight perfect matching in complete bipartite graph.

Processes / Algorithms / Pathways

Hopcroft-Karp: $O(\sqrt{V} \cdot E)$

- Find maximal set of vertex-disjoint shortest augmenting paths per phase.
- Each phase: BFS + DFS on directed graph.
- $O(\sqrt{V})$ phases.

Gale-Shapley (Stable Marriage): $O(n^2)$

- Women propose to men in order of preference. Men accept if free or prefer her.
- Returns women-optimal, men-pessimal stable matching.
- Multiple stable matchings can exist.

Hungarian Algorithm (Assignment Problem): $O(n^4) \rightarrow O(n^3)$

- Maintain feasible labeling: $l.h + r.h \geq w(l, r)$.
- Equality subgraph G_h : edges where $l.h + r.h = w(l, r)$.

- While no perfect matching: find augmenting path; if blocked, update labels.

Rules, Laws & Theorems

- **Lemma 25.1:** $M \oplus P$ is a matching with $|M|+1$ edges for M -augmenting path P .
- **Corollary 25.4 (Berge):** M is maximum iff no augmenting path exists.
- **Theorem 25.8:** Hopcroft-Karp runs in $O(\sqrt{V} \cdot E)$.
- **Theorem 25.9:** Gale-Shapley always terminates with stable matching.
- **Theorem 25.11:** Proposing side gets best possible partners; other side gets worst.
- **Theorem 25.14:** G_h with perfect matching gives optimal assignment.

Ch. 26 – Parallel Algorithms

Named Entities (Terms & Definitions)

- **Fork-join parallelism:** spawn and sync keywords; parallel for.
- **Work T_1 :** Total time on one processor.
- **Span T_∞ :** Fastest time on unlimited processors (critical path length).
- **Work law:** $T_p \geq T_1 / P$.
- **Span law:** $T_p \geq T_\infty$.
- **Parallelism:** T_1 / T_∞ (maximum possible speedup).
- **Greedy scheduler:** Assigns as many ready strands as possible each step.
- **Determinacy race:** Two parallel instructions access same memory, at least one writes.

Processes / Algorithms / Pathways

P-FIB(n): spawn P-FIB(n-1); P-FIB(n-2); sync.

- Work: $\Theta(\phi^n)$ (exponential). Span: $\Theta(n)$. Parallelism: $\Theta(\phi^n / n)$.

P-MERGE-SORT

- Spawn two recursive calls in parallel; use P-MERGE.
- Work: $\Theta(n \lg n)$. Span: $\Theta(\lg^3 n)$. Parallelism: $\Theta(n / \lg^2 n)$.

P-MERGE-AUX (parallel merge)

- Divide-and-conquer: pick median of larger subarray, binary search split point in smaller, place, recurse.
- Work: $\Theta(n)$. Span: $\Theta(\lg^2 n)$.

Formulas & Equations

- Work law: $T_p \geq T_\infty / P$
- Span law: $T_p \geq T_\infty$
- Speedup: $T_\infty / T_p \leq P$
- Parallelism: T_∞ / T_∞
- **Theorem 26.1:** Greedy scheduler: $T_p \leq T_\infty / P + T_\infty$
- **Corollary 26.2:** Greedy scheduler within factor 2 of optimal

Algorithm Parallelism Table

Algorithm	Work	Span	Parallelism
P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
P-MAT-VEC	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
P-MATRIX-MULTIPLY	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
P-MERGE-SORT	$\Theta(n \lg n)$	$\Theta(\lg^3 n)$	$\Theta(n/\lg^2 n)$

Ch. 27 – Online Algorithms

Named Entities

- **Online algorithm:** Receives input over time; decisions without knowing future.
- **Competitive ratio:** $\max\{A(I)/F(I) : I \in U\}$ (minimization); c-competitive if ratio $\leq c$.
- **Seer / optimal offline:** Knows entire future; benchmark.
- **Move-to-Front (MTF):** After search, move element to front.
- **Potential function:** $\Phi_i = 2 \cdot I(L^{\{MTF\}}_i, L^{\{OPT\}}_i)$ for MTF analysis.

Processes / Algorithms / Pathways

Elevator vs Stairs Problem

- **Hedge strategy:** Wait k minutes then take stairs. Competitive ratio = 2 (optimal).

Move-to-Front (MTF)

- Cost per operation: $2r(x) - 1$ (position r, search + swaps).
- **Theorem 27.1:** MTF competitive ratio = 4.
- Proof via potential function ($2 \cdot \text{inversions}$).

Caching Algorithms

- **LRU:** $O(k)$ competitive ratio.
- **FIFO:** $O(k)$ competitive ratio.
- **LIFO:** $\Theta(n/k)$ – unbounded.
- **Any deterministic:** $\Omega(k)$ lower bound.
- **RANDOMIZED-MARKING:** $O(\lg k)$ expected competitive ratio.

Comparisons & Trade-offs

Algorithm	Competitive Ratio	Notes
Always stairs	k	—
Always elevator	B/k	—
Hedge (wait k)	2	Optimal
MTF	4	List maintenance
LRU	$O(k)$	Caching
RANDOMIZED-MARKING	$O(\lg k)$	Best known

Ch. 28 – Matrix Operations

Named Entities

- **LUP decomposition:** $PA = LU$ (P = permutation, L = unit lower-triangular, U = upper-triangular).
- **Forward substitution:** Solve $Ly = Pb$ ($\Theta(n^2)$).
- **Back substitution:** Solve $Ux = y$ ($\Theta(n^2)$).
- **Schur complement:** $A' = vw^T/a_{11}$ (for LU step).
- **Symmetric positive-definite (SPD):** $A = A^T$ and $x^T A x > 0$ for all $x \neq 0$.
- **Least-squares approximation:** Minimize $\|y - Ax\|^2 = \|Ac - y\|^2$.

Processes / Algorithms / Pathways

LU-DECOMPOSITION(A, n): $\Theta(n^3)$. Fails if pivot = 0.

LUP-DECOMPOSITION(A, n): $\Theta(n^3)$. Pivoting for numerical stability.

- Find row with largest $|a_{ik}|$, swap rows.

Matrix Inversion via LUP: $\Theta(n^3)$

- Solve $AX_i = e_i$ for each column i .

Strassen-based Inversion: $O(M(n))$ where $M(n)$ = matrix multiplication time.

- Recursively invert SPD submatrices; Schur complement.

Least-Squares via Normal Equation: $c = (A^T A)^{-1} A^T y$

Rules, Laws & Theorems

Theorem 28.1: Matrix multiplication $\leq$ Inversion (if inversion takes $I(n)$, multiplication takes $O(I(n))$). **Theorem 28.2:** Inversion $\leq$ Multiplication (if multiplication takes $M(n)$, inversion takes $O(M(n))$). **Lemma 28.5 (Schur complement):** Schur complement of SPD matrix is SPD. **Corollary 28.6:** LU of SPD has all pivots > 0 .

Ch. 29 – Linear Programming

Named Entities

- **Linear programming:** Optimize linear objective subject to linear constraints.
- **Standard form:** $\max c^T x$ s.t. $Ax \leq b, x \geq 0$.
- **Feasible solution:** Satisfies all constraints.
- **Unbounded LP:** Feasible region unbounded; no finite optimal.
- **Dual LP:** $\min b^T y$ s.t. $A^T y \geq c, y \geq 0$ (for max primal).
- **Weak duality:** $c^T x \leq b^T y$ for any feasible primal x and dual y .
- **Strong duality:** $c^T x^* = b^T y^*$ at optimality.

Processes / Algorithms / Pathways

- **Simplex algorithm:** Moves along edges of feasible region. Exponential worst-case, fast in practice.
- **Ellipsoid algorithm:** First polynomial-time (Khachian 1979). $O(n^4 L)$. Slow in practice.
- **Interior-point methods:** Polynomial-time, practical (Karmarkar 1984).

LP Formulation Examples

- **Shortest paths:** maximize d_t subject to $d_s = 0, d_v \leq d_u + w(u,v)$.
- **Max flow:** maximize $\sum_v f\{sv\} - \sum_v f\{vs\}$ subject to capacity + conservation.
- **Minimum-cost flow:** minimize $\sum a(u,v) \cdot f_{uv}$ subject to flow constraints.

Rules, Laws & Theorems

Lemma 29.1 (Weak duality): $c^T x \leq b^T y$ for all feasible x, y .

Corollary 29.2: If $c^T x = b^T y$, then x, y are optimal.

Theorem 29.4 (Strong duality): If primal and dual are feasible and bounded, $c^T x^* = b^T y^*$. **Theorem 29.5:** Any LP in standard form has finite optimal, is infeasible, or is unbounded.

People & Dates

- Dantzig (1947): Simplex.
- Khachian (1979): Ellipsoid.
- Karmarkar (1984): Interior-point.
- Klee & Minty: exponential-time example for simplex.

Ch. 30 – Polynomials and the FFT

Named Entities

- **Polynomial:** $A(x) = \sum_{j=0}^{n-1} a_j x^j$. Degree-bound n .
- **Coefficient representation:** Vector $a = (a_0, a_1, \dots, a_{n-1})$.
- **Point-value representation:** n pairs $\{(x_k, y_k)\}$.
- **DFT (Discrete Fourier Transform):** $y_k = A(\omega_n^k)$ where $\omega_n = e^{2\pi i/n}$.
- **FFT (Fast Fourier Transform):** $\Theta(n \lg n)$ algorithm for DFT.
- **Convolution:** $(a \otimes b)_j = \sum_k a_k \cdot b_{j-k}$.

Processes / Algorithms / Pathways

FFT (Cooley-Tukey)

```
FFT(a, n):
1 if n == 1: return a
2  $\omega_n = e^{2\pi i/n}$ ;  $\omega = 1$ 
3  $a_{\text{even}} = (a_0, a_2, \dots, a_{n-2})$ 
4  $a_{\text{odd}} = (a_1, a_3, \dots, a_{n-1})$ 
5  $y_{\text{even}} = \text{FFT}(a_{\text{even}}, n/2)$ 
6  $y_{\text{odd}} = \text{FFT}(a_{\text{odd}}, n/2)$ 
7 for  $k = 0$  to  $n/2-1$ :
8      $y_k = y_{\text{even}_k} + \omega \cdot y_{\text{odd}_k}$ 
9      $y_{k+n/2} = y_{\text{even}_k} - \omega \cdot y_{\text{odd}_k}$ 
10     $\omega = \omega \cdot \omega_n$ 
11 return  $y$ 
```

- **Recurrence:** $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \lg n)$.
- Butterfly operation: $y_k = \text{even} + \omega \cdot \text{odd}$; $y_{k+n/2} = \text{even} - \omega \cdot \text{odd}$.

Polynomial Multiplication via FFT (4 steps)

1. Double degree-bound (add n zero coefficients).
2. Compute DFT (FFT) of order $2n$ on each polynomial.
3. Pointwise multiply: $C(\omega_{2n}^k) = A(\omega_{2n}^k) \cdot B(\omega_{2n}^k)$.
4. Apply inverse FFT. $\rightarrow \Theta(n \lg n)$.

Rules, Laws & Theorems

- **Lemma 30.3 (Cancellation lemma):** $\omega_{dn}^{dk} = \omega_n^k$.
- **Lemma 30.5 (Halving lemma):** Squares of n -th roots = $(n/2)$ -th roots.
- **Lemma 30.6 (Summation lemma):** $\sum_{j=0}^{n-1} (\omega_n^k)^j = 0$ for k not divisible by n .
- **Theorem 30.8 (Convolution theorem):** $\text{DFT}_{2n}(a \otimes b) = \text{DFT}_{2n}(a) \cdot \text{DFT}_{2n}(b)$.

Representations Comparison

Representation Evaluation		Addition Multiplication	
Coefficient	$\Theta(n)$ (Horner)	$\Theta(n)$	$\Theta(n^2)$ naive; $\Theta(n \lg n)$ via FFT
Point-value	Interpolation needed	$\Theta(n)$	$\Theta(n)$

Dates & People

- Cooley & Tukey (1965): FFT. Gauss (1805): earliest known.
- Frigo & Johnson: FFTW library.

Ch. 31 – Number-Theoretic Algorithms

Named Entities (Terms & Definitions)

- **GCD**: Greatest common divisor of a and b ($\gcd(a,b)$).
- **Modular arithmetic**: $a \equiv b \pmod{n}$ if $n \mid (a-b)$.
- **Multiplicative inverse**: $a^{-1} \pmod{n}$ exists iff $\gcd(a,n) = 1$.
- **Group $\mathbb{Z}_n^*$** : $\{a \in \mathbb{Z}_n : \gcd(a,n) = 1\}$; multiplicative group modulo n .
- **Euler's totient $\phi(n)$** : $|\mathbb{Z}_n^*|$.
- **Primitive root**: Generator of $\mathbb{Z}_n^*$.
- **RSA**: Rivest-Shamir-Adleman public-key cryptosystem.
- **Chinese Remainder Theorem**: System of congruences with pairwise coprime moduli.

Processes / Algorithms / Pathways

EUCLID(a, b) – GCD: $O(\lg b)$

```
EUCLID( $a, b$ )
1 if  $b == 0$ : return  $a$ 
2 else return EUCLID( $b, a \bmod b$ )
```

- Lamé's theorem: EUCLID takes $O(\lg b)$ divisions.

EXTENDED-EUCLID(a, b) – GCD + Bézout coefficients: $O(\lg b)$

```
EXTENDED-EUCLID( $a, b$ )
1 if  $b == 0$ : return ( $a, 1, 0$ )
2 else ( $d, x, y$ ) = EXTENDED-EUCLID( $b, a \bmod b$ )
3     return ( $d, y, x - \lfloor a/b \rfloor \cdot y$ )
```

- Returns (d, x, y) such that $d = \gcd(a,b) = ax + by$.

Modular Linear Equations

- Solve $ax \equiv b \pmod{n}$. Solutions exist iff $d = \gcd(a,n)$ divides b . Then d solutions: $x_0 + t(n/d)$ for $t = 0, 1, \dots, d-1$.

Chinese Remainder Theorem

- $n = n_1 \cdot n_2 \cdot \dots \cdot n_k$ where n_i pairwise coprime.
- System $x \equiv a_i \pmod{n_i}$ has unique solution modulo n .
- $x = \sum a_i \cdot c_i \cdot i$ where $i = n/n_i$, $c_i = i^{-1} \pmod{n_i}$.

RSA Public-Key Cryptosystem

1. Choose large primes p, q . Compute $n = pq$, $\phi(n) = (p-1)(q-1)$.
2. Choose e with $\gcd(e, \phi(n)) = 1$. Compute $d = e^{-1} \pmod{\phi(n)}$.
3. Public key: (e, n) . Private key: d .
4. Encrypt: $c = m^e \pmod{n}$. Decrypt: $m = c^d \pmod{n}$.
5. Security relies on difficulty of factoring n .

PRIMALITY TESTING (Miller-Rabin)

- Probabilistic test. Uses witness a : if $a^{n-1} \not\equiv 1 \pmod{n}$ or finds nontrivial square root of 1, n is composite.
- For odd $n > 2$: write $n-1 = 2^s \cdot t$ (t odd). Check $a^t, a^{2t}, a^{4t}, \dots, a^{2^{s-1}t} \pmod{n}$.
- If n is composite, at least $3/4$ of $a \in \mathbb{Z}_n^*$ are witnesses.
- Error probability $< 4^{-k}$ after k independent trials.

Formulas & Equations

- $\gcd(a, b) = \gcd(b, a \bmod b)$
- Bézout: $\exists x, y: ax + by = \gcd(a, b)$
- Fermat: $a^{p-1} \equiv 1 \pmod{p}$ for prime $p \nmid a$
- Euler: $a^{\phi(n)} \equiv 1 \pmod{n}$ for $\gcd(a, n) = 1$
- RSA: $m^{ed} \equiv m \pmod{n}$ where $ed \equiv 1 \pmod{\phi(n)}$

Ch. 32 – String Matching

Named Entities

- **Pattern P**: Length m , text T : length n . Find all occurrences of P in T .
- **Suffix**: $T[i..j]$ where $j = n$. **Prefix**: $T[1..i]$.
- **Suffix array**: Lexicographically sorted suffixes of T .
- **LCP array**: Longest common prefix between consecutive suffixes in suffix array.

Processes / Algorithms / Pathways

Naive String Matching: $O((n-m+1)m)$

- Slide pattern over text, compare character by character.

Rabin-Karp: $O(n)$ average, $O(nm)$ worst

- Hash pattern. Compute rolling hash for each text substring.

- Hash collision resolved by checking equality.
- Rolling hash: $h(T[s+1..s+m]) = (d \cdot (h(T[s..s+m-1]) - T[s+1] \cdot d^{\{m-1\}}) + T[s+m+1]) \bmod q$.

String Matching with Finite Automata: $O(n + m|\Sigma|)$

- Build transition function $\delta(q, a)$ for pattern.
- Preprocessing: $O(m|\Sigma|)$. Matching: $\Theta(n)$.

KMP (Knuth-Morris-Pratt): $O(n + m)$

- Compute prefix function π for pattern: length of longest proper prefix that is also suffix.
- Matching: shift pattern using π without backing up text pointer.
- Never re-compares characters matched earlier.

Suffix Arrays: $O(n)$ construction

- SA: array of start indices of all suffixes of T in lexicographic order.
- LCP: longest common prefix between $SA[i]$ and $SA[i-1]$.
- Pattern search: binary search on SA using LCP to accelerate $\rightarrow O(m + \lg n)$.

Comparisons & Trade-offs

Algorithm	Preprocessing	Matching	Space
Naive	None	$O((n-m+1)m)$	$O(1)$
Rabin-Karp	$\Theta(m)$	$O(n)$ avg, $O(nm)$ worst	$O(1)$
Finite automaton	$O(m \Sigma)$	Σ	$O(m \Sigma)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(m)$

Ch. 33 – Machine-Learning Algorithms

Named Entities

- **Clustering**: Partition points into groups by similarity.
- **k-median**: Minimize sum of distances from points to nearest cluster center.
- **k-means**: Minimize sum of squared distances.
- **k-means++**: Improved initialization for k-means.
- **Multiplicative weights**: Weighted majority / Hedge algorithm.
- **Gradient descent**: Iteratively move in direction of steepest descent (negative gradient).
- **Loss function**: Measures error of current parameters.

Processes / Algorithms / Pathways

k-means Clustering

- Initialize k centers. Repeat: assign each point to nearest center; recompute centers as mean of assigned points.
- Hill-climbing: converges to local optimum.
- k-means++: pick first center uniformly; subsequent centers with probability $\propto \text{distance}^2$.

Multiplicative Weights (Weighted Majority)

- Maintain weights on experts. Predict by weighted majority vote.
- Update: decrease weight of wrong experts by factor $(1-\epsilon)$.
- **Theorem:** Number of mistakes $\leq (1+\epsilon) \cdot (\text{best expert mistakes}) + O(\log(n)/\epsilon)$.

Gradient Descent

- $\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$ where η = learning rate.
- Works for convex loss functions (guaranteed global min).
- Stochastic gradient descent (SGD): use single sample's gradient estimate.

Ch. 34 – NP-Completeness

Named Entities

- **P:** Problems solvable in polynomial time.
- **NP:** Problems verifiable in polynomial time (certificate).
- **NP-complete (NPC):** NP-hard and in NP.
- **NP-hard:** All NP problems reduce to it.
- **Polynomial-time reduction:** f such that $x \in A$ iff $f(x) \in B$; f computable in poly time.
- **Certificate:** Proof that answer is YES, verifiable in poly time.

Processes / Algorithms / Pathways

- To prove problem B is NP-complete:
 1. Show $B \in \text{NP}$ (polynomial-time verifiable).
 2. Reduce known NP-complete problem A to B ($A \leq_P B$).
 3. Show reduction is polynomial time.

Standard NP-Complete Problems (reduction chain)

- **CIRCUIT-SAT** $\rightarrow$ SAT $\rightarrow$ 3-CNF-SAT $\rightarrow$ CLIQUE $\rightarrow$ VERTEX-COVER $\rightarrow$ HAM-CYCLE $\rightarrow$ TSP
- SAT $\rightarrow$ SUBSET-SUM

Well-known NP-Complete Problems

- **SAT**: Boolean formula satisfiability.
- **3-CNF-SAT**: Conjunctive normal form, each clause exactly 3 literals.
- **CLIQUE**: Does graph contain clique of size k ?
- **VERTEX-COVER**: Does graph have vertex cover of size k ?
- **HAM-CYCLE**: Does graph have Hamiltonian cycle?
- **TSP**: Traveling salesperson with total distance $\leq D$.
- **SUBSET-SUM**: Does subset sum to target t ?

Rules, Laws & Theorems

Cook-Levin Theorem (1971): CIRCUIT-SAT is NP-complete. First NP-complete problem. If any NP-complete problem has polynomial-time algorithm, then $P = NP$.

Edge Cases & Common Pitfalls

- Some problems seem NP-complete but are in P for restricted inputs (e.g., 2-CNF-SAT).
- Proving NP-completeness via wrong direction: must reduce FROM known NPC TO new problem.
- $NP \neq$ "not polynomial" – it's nondeterministic polynomial.

Ch. 35 – Approximation Algorithms

Named Entities

- **Approximation algorithm**: Guarantees solution within factor ρ of optimal.
- **ρ -approximation algorithm**: $|SOL| \leq \rho \cdot |OPT|$ for minimization ($SOL/OPT \leq \rho$).
- **PTAS (Polynomial-Time Approximation Scheme)**: $(1+\epsilon)$ -approximation for any $\epsilon > 0$.
- **FPTAS (Fully PTAS)**: Time polynomial in n and $1/\epsilon$.

Processes / Algorithms / Pathways

Vertex-Cover Approximation (2-approximation)

- Take edge (u,v) , add both u,v to cover, remove all incident edges. Repeat.

TSP Approximation

- **With triangle inequality**: MST-based (double-tree) gives 2-approx; Christofides gives 1.5-approx.
- **Without triangle inequality**: No polynomial-time ρ -approximation for any ρ (unless $P=NP$).

Set-Cover Approximation (greedy)

- Repeatedly pick set that covers most uncovered elements.
- $H(d)$ -approximation where $d = \max \text{ set size}$. $H(d) = 1 + 1/2 + \dots + 1/d = \ln d + O(1)$.

Subset-Sum (FPTAS)

- Dynamic programming with scaling: discard low-order bits to get polynomial time.
- $O(n^2/\epsilon)$ time; produces solution $\geq (1-\epsilon) \cdot \text{OPT}$.

Comparisons & Trade-offs

Problem	Approximation	Method
Vertex cover	2-approximation	Maximal matching
TSP (triangle inequality)	1.5-approximation (Christofides)	MST + perfect matching
Set cover	$H(d)$ -approximation	Greedy
Subset-sum	FPTAS $(1-\epsilon)$	Scaled DP
Max clique	No constant factor (unless $P=NP$)	—

Appendix A – Summations

Named Entities

- **Arithmetic series:** $\sum_{k=1}^n k = n(n+1)/2$.
- **Geometric series:** $\sum_{k=0}^n x^k = (x^{n+1}-1)/(x-1)$ for $x \neq 1$.
- **Infinite decreasing geometric series:** $\sum_{k=0}^{\infty} x^k = 1/(1-x)$ for $|x| < 1$.
- **Harmonic series:** $H_n = \sum_{k=1}^n 1/k = \ln n + \gamma + O(1/n)$. $\gamma \approx 0.57721$ (Euler's constant).
- **Telescoping series:** $\sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_0$.

Formulas & Equations

- $\sum k = n(n+1)/2$
- $\sum k^2 = n(n+1)(2n+1)/6$
- $\sum k^3 = n^2(n+1)^2/4$
- $\sum_{k=0}^{\infty} k x^k = x/(1-x)^2$ for $|x| < 1$
- $\sum_{k=0}^{\infty} k^2 x^k = x(1+x)/(1-x)^3$

Bounding Techniques

1. **Induction:** Prove base, assume for n , prove $n+1$. Must use same constant for all n .
2. **Bounding terms:** $\sum a_k \leq n \cdot a_{\max}$. Ratio method if $a_{k+1}/a_k \leq r < 1$.

3. **Splitting sums:** Partition range into pieces; bound each.
4. **Integral approximation:** $\int_{\{0\}^{\{n\}} f(x)dx \leq \sum_{k=1}^n f(k) \leq \int_{\{1\}^{\{n+1\}} f(x)dx$ for monotone increasing f . Reverse for decreasing f .

Appendix B – Sets, Etc.

Sets

- **Operations:** $\cup$ (union), $\cap$ (intersection), $-$ (difference), complement.
- **DeMorgan's laws:** $A \cap B = \bar{A} \cup \bar{B}$, $A \cup B = \bar{\bar{A} \cap \bar{B}}$.
- **Principle of inclusion-exclusion:** $|\cup A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots + (-1)^{n-1} |\cap A_i|$.

Relations

- **Equivalence relation:** Reflexive + Symmetric + Transitive. Equivalence classes form partition.
- **Partial order:** Reflexive + Antisymmetric + Transitive.
- **Total order:** Partial order that is total.

Functions

- **Injection (one-to-one):** $a \neq a' \Rightarrow f(a) \neq f(a')$.
- **Surjection (onto):** Range = Codomain.
- **Bijection:** Both injective and surjective.

Graphs (see also Ch 20)

- **Connected:** Every vertex reachable from all others.
- **Free tree:** Connected + acyclic + $|E| = |V| - 1$.
- **Theorem B.2:** Six equivalent definitions of free tree.

Trees

- **Rooted tree:** Distinguished root. Parent, child, ancestor, descendant.
- **Binary tree:** Left/right children (position matters).
- **Full binary tree:** Every node has 0 or 2 children.
- **Complete k-ary tree:** Leaves at same depth; internal nodes have k children.
- **Kraft inequality:** $\sum_{\text{leaves}} 2^{-\text{depth}} \leq 1$.

Appendix C – Counting and Probability

Counting

- k -strings over n -set: n^k
- Permutations: $n!$
- k -permutations: $n!/(n-k)!$

- **k-combinations:** $C(n, k) = n! / (k!(n-k)!)$
- **Binomial theorem:** $(x+y)^n = \sum_{k=0}^n C(n, k) x^k y^{n-k}$
- **Stirling upper bound:** $C(n, k) \leq (en/k)^k$

Probability Axioms

1. $\Pr\{A\} \geq 0$
2. $\Pr\{S\} = 1$
3. $\Pr\{\cup A_i\} = \sum \Pr\{A_i\}$ for disjoint events

Random Variables

- **Expectation:** $E[X] = \sum x \cdot \Pr\{X=x\}$. Linearity: $E[X+Y] = E[X] + E[Y]$.
- **Variance:** $\text{Var}[X] = E[(X-\mu)^2] = E[X^2] - E^2[X]$.
- **Markov's inequality:** $\Pr\{X \geq t\} \leq E[X]/t$.
- **Jensen's inequality:** $f(E[X]) \leq E[f(X)]$ for convex f .

Distributions

- **Geometric:** $\Pr\{X=k\} = (1-p)^{k-1}p$. $E[X] = 1/p$. $\text{Var} = (1-p)/p^2$.
- **Binomial:** $b(k; n, p) = C(n, k) p^k (1-p)^{n-k}$. $E[X] = np$. $\text{Var} = np(1-p)$.
- **Chernoff bound:** $\Pr\{X \geq r\} < (np/r)^r e^{r-np}$ for $r > np$.

Appendix D – Matrices

Named Entities

- **Matrix:** Rectangular array $A = (a_{ij}) \in \mathbb{R}^{m \times n}$.
- **Transpose:** $(A^T)_{ij} = a_{ji}$.
- **Square matrix types:** diagonal, identity, tridiagonal, upper/lower-triangular, permutation, symmetric.
- **Determinant:** $\det(A) = \sum a_{ij} \cdot (-1)^{i+j} \cdot \det(A_{ij})$.
- **Rank:** Maximum number of linearly independent rows/columns.
- **Positive-definite:** $x^T A x > 0$ for all $x \neq 0$.

Matrix Operations

- **Addition:** $C = A + B$ ($c_{ij} = a_{ij} + b_{ij}$).
- **Multiplication:** $(AB)_{ij} = \sum_k a_{ik} b_{kj}$. NOT commutative.
- **Inner product:** $\langle x, y \rangle = x^T y = \sum x_i y_i$.
- **Outer product:** $(xy^T)_{ij} = x_i y_j$.

Properties

- **Theorem D.1:** Square matrix has full rank $\iff$ nonsingular (inverse exists).
- **Theorem D.4:** $\det(AB) = \det(A) \cdot \det(B)$. Row swap flips sign.
- **Theorem D.5:** A is singular $\iff \det(A) = 0$.

- **Theorem D.6:** $A^T A$ is positive-definite if A has full column rank.
-

Cross-Cutting Topics

Design Paradigms & Meta-Methods

1. **Divide and Conquer** – (Ch 2, 4) Split problem into independent subproblems, solve recursively, combine. Examples: merge sort, Strassen, FFT. Recurrence: $T(n) = aT(n/b) + f(n)$. Master theorem solves in 3 cases.
2. **Dynamic Programming** – (Ch 14) Overlapping subproblems + optimal substructure. Two implementations: top-down with memoization (recursion + table) and bottom-up (iterative, all subproblems). Four-step method: (1) characterize structure, (2) define recursively, (3) compute value, (4) construct solution. Examples: rod cutting ($\Theta(n^2)$), matrix-chain ($\Theta(n^3)$), LCS ($\Theta(mn)$), OBST ($\Theta(n^3)$).
3. **Greedy Algorithms** – (Ch 15) Greedy-choice property + optimal substructure. Make locally optimal choice first, leaving one subproblem. Examples: activity selection ($\Theta(n)$), Huffman codes ($\Theta(n \lg n)$), fractional knapsack, offline caching. Proves optimality via exchange argument.
4. **Amortized Analysis** – (Ch 16) Average per-operation cost over worst-case sequence. Three methods: aggregate ($T(n)/n$), accounting (charge + credit), potential (Φ function). Examples: MULTIPOP, binary counter ($O(1)$ amortized per INCREMENT), dynamic tables ($O(1)$ amortized per INSERT).
5. **Randomization** – (Ch 5, 7, 9) Uses random number generator in algorithm. Benefits: no worst-case input (randomized quicksort), expected linear time (RANDOMIZED-SELECT), probabilistic primality testing. Analysis uses indicator random variables + linearity of expectation.
6. **Fork-Join Parallelism** – (Ch 26) spawn/sync keywords. Work T_1 (sequential), span T_∞ (critical path). Greedy scheduling: $T_p \leq T_1/P + T_\infty$. Parallelism = T_1/T_∞ .
7. **Online Algorithms** – (Ch 27) Input arrives over time. Competitive analysis vs optimal offline. Examples: MTF (competitive ratio 4), LRU ($O(k)$), RANDOMIZED-MARKING ($O(\lg k)$).
8. **Augmenting Data Structures** – (Ch 17) Add fields, maintain invariants, implement new operations. Theorem 17.1: if field computable from children in $O(1)$ and rotation-maintainable in $O(1)$, RB operations stay $O(\lg n)$. Examples: order-statistic trees (size), interval trees (max).

Proof & Argument Patterns

1. **Loop Invariants** – Used for iterative algorithm correctness. Three parts: (a) Initialization: invariant holds before first iteration; (b) Maintenance: if holds before iteration, holds after; (c) Termination: invariant + loop condition gives correctness. Classic: INSERTION-SORT, PARTITION, HEAPSORT, BUILD-MAX-HEAP.
2. **Induction** – For recursive algorithms and structural properties. Base case + inductive step. Uses: BST inorder walk ($\Theta(n)$), red-black tree height bound (Lemma 13.1), free tree properties (Theorem B.2).
3. **Substitution Method** – For solving recurrences. (1) Guess form; (2) verify by induction; (3) solve for constants. Example: $T(n) = 2T(n/2) + n$, guess $T(n) \leq cn \lg n$.
4. **Recursion Tree Method** – Draw tree where each node = cost of subproblem. Sum costs per level, total across levels. Useful for guessing form for substitution method.
5. **Cut-and-Paste** – For optimal substructure. Assume subproblem solution is not optimal; cut it out and paste in optimal one; derive contradiction with optimality of original solution. Used in DP (rod cutting, LCS).
6. **Exchange Argument** – For greedy correctness. Take any optimal solution; swap non-greedy element with greedy element without worsening solution; iteratively transform to greedy solution. Used in activity selection (Theorem 15.1), Huffman codes (Lemma 15.2).
7. **Cut Property** – MST correctness: any light edge crossing cut that respects A is safe for A (Theorem 21.1). Used to prove Kruskal and Prim.
8. **Max-Flow Min-Cut Theorem** (24.6): Three equivalent statements – f is max flow; G_f has no augmenting path; $|f| = c(S,T)$ for some cut. Foundation of flow theory.
9. **Potential Method** – For amortized analysis. Φ maps data structures to reals. $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Total cost = $\sum c_i + \Phi(D_n) - \Phi(D_0)$. Example: dynamic table with $\Phi = 2 \cdot \text{num} - \text{size}$.
10. **Reduction** – For NP-completeness. Given known NP-complete A , reduce $A \leq_P B$ to show B is NP-complete. Must reduce FROM known NPC TO new problem.
11. **Parenthesis Theorem** (20.7) – DFS intervals: for any u, v , intervals are either disjoint or one contains the other. White-path theorem (20.9): v is descendant of u iff white-path exists at $u.d$.

12. **Pigeonhole Principle** – Used in: hashing collisions, comparison sort lower bound, Hall's theorem.

Probability & Statistics Foundation

(from Appendix C, Ch 5)

- **Indicator random variables:** $E[I\{A\}] = \Pr\{A\}$. Sum of indicators $\rightarrow$ expected count.
- **Linearity of expectation:** $E[X+Y] = E[X] + E[Y]$. Holds even if X, Y dependent. Critical for analyzing randomized algorithms.
- **Variance:** $\text{Var}[X] = E[X^2] - E^2[X]$. Standard deviation $\sigma = \sqrt{\text{Var}}$.
- **Bernoulli trial:** Single success/failure experiment. $E[X] = p$, $\text{Var}[X] = p(1-p)$.
- **Binomial distribution:** $b(k; n, p) = C(n, k) p^k (1-p)^{n-k}$. k successes in n independent Bernoulli trials. $E[X] = np$. $\text{Var}[X] = np(1-p)$.
- **Geometric distribution:** $\Pr\{X=k\} = (1-p)^{k-1}p$. Number of trials until first success. $E[X] = 1/p$. $\text{Var}[X] = (1-p)/p^2$.
- **Markov's inequality:** $\Pr\{X \geq t\} \leq E[X]/t$ for nonnegative X . Weak but general.
- **Chernoff bound:** $\Pr\{X \geq r\} < (np/r)^r e^{r-np}$ for $r > np$ (binomial). Strong exponential tail bound.
- **Union bound (Boole's inequality):** $\Pr\{\bigcup A_i\} \leq \sum \Pr\{A_i\}$. Used in: hashing analysis, randomized algorithm error bounds.
- **Bayes' theorem:** $\Pr\{A|B\} = \Pr\{A\} \cdot \Pr\{B|A\} / \Pr\{B\}$. Updated beliefs given evidence.
- **Conditional probability:** $\Pr\{A|B\} = \Pr\{A \cap B\} / \Pr\{B\}$.
- **Jensen's inequality:** $f(E[X]) \leq E[f(X)]$ for convex f . Used for lower bounds.

Mnemonics & Memory Aids

- **Big-O ordering:** "1 < lg n < n < n lg n < n² < n³ < 2ⁿ < n!" (constant-log-linear-linearithmic-quadratic-cubic-exponential-factorial)
- **Master Theorem Cases:** Compare $f(n)$ with $n^{\log_b a}$. $f(n)$ is smaller $\rightarrow$ case 1 ($\Theta(n^{\log_b a})$). $f(n)$ is equal $\rightarrow$ case 2 ($\Theta(n^{\log_b a} \lg n)$). $f(n)$ is larger and regular $\rightarrow$ case 3 ($\Theta(f(n))$).
- **DFS Edge Colors:** "White $\rightarrow$ Tree, Gray $\rightarrow$ Back, Black $\rightarrow$ Forward/Cross"
- **Red-Black Properties:** "Root-Black, Leaves-Black, No-Red-Red, Equal-Black-Count" (RB-5: root-black, leaf-black, red-no-red-children, equal-black-paths)
- **DP Steps:** "Characterize, Define, Compute, Construct" (4-step method)
- **Greedy Requirements:** "Greedy Choice + Optimal Substructure" (both required)
- **Amortized Methods:** "Aggregate (total/n), Accounting (charge + credit), Potential (Φ function)"

- **Graph Algorithm Times:** $BFS = O(V+E)$, $DFS = O(V+E)$, $TopSort = O(V+E)$, $SCC = O(V+E)$, $Dijkstra = O(E \lg V)$, $Bellman-Ford = O(VE)$, $Prim = O(E \lg V)$, $Kruskal = O(E \lg V)$, $Floyd-Warshall = O(V^3)$
- **NP-Completeness Chain:** "CIRCUIT-SAT $\rightarrow$ SAT $\rightarrow$ 3-CNF-SAT $\rightarrow$ CLIQUE $\rightarrow$ VERTEX-COVER $\rightarrow$ HAM-CYCLE $\rightarrow$ TSP" and "SAT $\rightarrow$ SUBSET-SUM"
- **Flow Conservation:** "Flow In = Flow Out" (for all nodes except source/sink)
- **Five Sorting Algorithms:** "Insertion $O(n^2)$, Merge $O(n \lg n)$, Heap $O(n \lg n)$, Quick $O(n^2)$ worst $O(n \lg n)$ avg, Counting $O(n+k)$ " – know which are comparison-based, stable, in-place

People & Dates

- **C.A.R. Hoare** (1960/1962): Quicksort
- **John von Neumann:** Mergesort
- **Edsger Dijkstra** (1959): Dijkstra's algorithm
- **Robert Floyd:** Floyd-Warshall (1962)
- **J.B.J. Fourier / Cooley & Tukey** (1965): FFT; Gauss found it first (1805)
- **Rivest, Shamir, Adleman** (1977): RSA cryptosystem
- **Dantzig** (1947): Simplex algorithm for LP
- **Khachian** (1979): Ellipsoid algorithm (first poly-time LP)
- **Karmarkar** (1984): Interior-point method for LP
- **Cook, Karp, Levin** (1971-1972): NP-completeness theory
- **Cook** (1971): Cook-Levin theorem (SAT is NP-complete)
- **Hopcroft & Karp** (1973): $O(\sqrt{V} \cdot E)$ bipartite matching
- **Gale & Shapley** (1962): Stable marriage algorithm
- **Sleator & Tarjan:** Amortized analysis, splay trees, competitive analysis (MTF)
- **Bayer & McCreight:** B-trees
- **Kruskal** (1956): MST algorithm
- **Prim / Jarník** (~1930/1957): MST algorithm
- **Bellman & Ford** (1958): Bellman-Ford shortest paths
- **Knuth, Morris, Pratt** (1977): KMP string matching
- **Carter & Wegman** (1979): Universal hashing
- **Huffman** (1952): Huffman coding
- **Strassen** (1969): $O(n^{\lg 7})$ matrix multiplication
- **Thorup:** $O(V+E)$ undirected shortest paths (integer weights)
- **Cormen, Leiserson, Rivest, Stein:** CLRS textbook authors

Exam Questions by Type

MCQ

1. **Q:** What is the worst-case running time of quicksort?
A: $O(n^2)$.
Distractor: $O(n \lg n)$ – that's average-case.
Distractor: $O(n)$ – only if input already sorted in best-case for some variants.

2. **Q:** Which data structure gives $O(1)$ amortized insert and $O(\lg n)$ extract-max?
A: Binary heap.
Distractor: BST – $O(h)$ where h can be $O(n)$ in worst case.
3. **Q:** The Floyd-Warshall algorithm solves which problem?
A: All-pairs shortest paths.
Distractor: Single-source shortest paths – that's Dijkstra.
Distractor: Minimum spanning tree – that's Prim/Kruskal.
4. **Q:** A red-black tree with n internal nodes has height at most:
A: $2 \lg(n+1)$.
Distractor: $\lg(n+1)$ – that's the height bound for AVL trees.
Distractor: n – a red-black tree never degenerates to linear chain.
5. **Q:** Which of the following is NOT a comparison sort?
A: Counting sort.
Distractor: Heapsort, mergesort, quicksort – all are comparison sorts.
6. **Q:** The master theorem case 2 applies when $f(n) = \Theta(n^{\{\log_b a\}} \lg^k n)$. What is the solution?
A: $T(n) = \Theta(n^{\{\log_b a\}} \lg^{\{k+1\}} n)$.
Distractor: $T(n) = \Theta(n^{\{\log_b a\}} \lg^k n)$ – missing $+1$ in the $\lg$ exponent.
7. **Q:** In a flow network, the max flow value equals:
A: The capacity of the minimum cut.
Distractor: The capacity of the maximum cut – opposite of min cut.
8. **Q:** What is the competitive ratio of LRU caching?
A: $O(k)$.
Distractor: $O(\lg k)$ – that's RANDOMIZED-MARKING.
Distractor: $\Theta(n/k)$ – that's LIFO.
9. **Q:** Which property must a relation satisfy to be a partial order?
A: Reflexive + Antisymmetric + Transitive.
Distractor: Reflexive + Symmetric + Transitive – that's equivalence relation.
10. **Q:** What is the amortized cost of TABLE-INSERT using the accounting method?
A: 3.
Distractor: 1 – that's the actual cost of a non-expanding insert.
Distractor: 2 – doesn't account for reinsertion of existing items.

Short Answer

- Q:** State the three properties a relation must satisfy to be an equivalence relation.
Rubric: Reflexive (1pt): $a R a$ for all a . Symmetric (1pt): $a R b \implies b R a$. Transitive (1pt): $a R b \implies b R c \implies a R c$.
- Q:** What is the difference between DP and divide-and-conquer?
Rubric: DP has overlapping subproblems (2pt); D&C has disjoint subproblems (2pt). Both require optimal substructure (1pt). DP saves results in table; D&C recomputes.
- Q:** Define the greedy-choice property.
Rubric: A globally optimal solution can be assembled by making locally optimal choices (3pt). Must prove greedy choice is part of some optimal solution (2pt).
- Q:** What does it mean for a problem to be NP-complete?
Rubric: (1) Problem is in NP – verifiable in polynomial time (2pt). (2) Problem is NP-hard – all NP problems reduce to it in polynomial time (3pt).
- Q:** Explain the three methods of amortized analysis.
Rubric: Aggregate – compute total cost $T(n)$, divide by n (2pt). Accounting – assign differing amortized costs, maintain nonnegative credit (2pt). Potential – Φ function; $\hat{c}_i = c_i + \Delta\Phi$ (1pt).
- Q:** When can a hash table use open addressing?
Rubric: Load factor $\alpha \leq 1$ (1pt). Deletion requires care – use DELETED marker or special linear probing delete (2pt). Must guarantee probe sequence covers all slots (2pt).

Trace / Apply

- Input:** $A = [5, 2, 4, 6, 1, 3]$. Apply INSERTION-SORT step by step.
Expected output: $[1, 2, 3, 4, 5, 6]$.
Key invariant: After processing $A[j]$, $A[1..j]$ is sorted.
- Input:** Graph edges $(s,a,10), (s,b,5), (a,b,2), (a,t,1), (b,a,3), (b,t,9)$. Run Dijkstra from s .
Expected: $s.d=0, b.d=5, a.d=7$ (via $s \rightarrow b \rightarrow a$), $t.d=8$ (via $s \rightarrow b \rightarrow a \rightarrow t$).
Why: b relaxes first (extract-min $b=5$). Then (b,a) relaxes $a.d$ from $10 \rightarrow 7$. Then (a,t) relaxes $t.d$ from $9 \rightarrow 8$.
- Input:** $X = \text{ABCB DAB}$, $Y = \text{BDCABA}$. Find LCS.
Expected: "BCBA", "BDAB", or "BCAB" – all length 4.
Method: Fill LCS table $c[1..m, 1..n]$; trace back via arrows.
- Input:** Items $(w,v): (10,60), (20,100), (30,120)$. Capacity $W=50$. Solve fractional knapsack.
Expected: Item 1 ($10/60$) all, item 2 ($20/100$) all, item 3

(20/80) 2/3. Total = 240.

Why: Sort by v/w: item1=6, item2=5, item3=4. Take greedily.

5. **Trace HEAPSORT** on $A = [5, 13, 2, 25, 7, 17, 20, 8, 4]$.
Expected after BUILD-MAX-HEAP: $[25, 13, 20, 8, 7, 17, 2, 5, 4]$.
Final sorted: $[2, 4, 5, 7, 8, 13, 17, 20, 25]$.
6. **Trace PARTITION** on $A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$.
Expected output: pivot at position 8 with value 11.
Low side: $[9, 5, 8, 7, 4, 2, 6]$. **High side:** $[21, 13, 19, 12]$.

Diagram Label

1. **Diagram:** Complete red-black tree. **Label:** Root (black), red nodes (2 children black), NIL leaves (black), black-height. Show that longest path (alternating red-black) $\leq 2 \times$ shortest (all black).
2. **Diagram:** Flow network with source s , sink t . **Label:** Capacity, flow, residual edges, augmenting path, cut (S, T) . Show max-flow = min-cut.
3. **Diagram:** Decision tree for comparison sort on 3 elements. **Label:** Internal node = comparison, leaf = permutation. Show $3! = 6$ leaves, height $\geq \lg(3!) = \Omega(\lg 6) = \Omega(3 \lg 3)$.
4. **Diagram:** BFS tree. **Label:** Source s , distances d , queue Q with gray vertices. Show waves of discovery.
5. **Diagram:** Red-black tree insertion. **Label:** Cases 1, 2, 3. Show rotation and recoloring steps.

Essay / Long-Form

1. **Q:** Compare and contrast dynamic programming and greedy algorithms. When would you use each?
Key points: Both require optimal substructure. DP has overlapping subproblems (solved once, cached); greedy makes locally optimal choice leaving one subproblem. DP: bottom-up or top-down memoization. Greedy: prove greedy-choice property via exchange argument. Examples: 0-1 knapsack (DP) vs fractional knapsack (greedy). DP generally $O(n^2)$ or $O(n^3)$; greedy $O(n)$ or $O(n \lg n)$. Choose DP when overlapping subproblems and no greedy choice exists.
2. **Q:** Explain NP-completeness and its implications for algorithm design.
Key points: P = poly-time decidable; NP = poly-time verifiable; NPC = NP-hard + NP. Cook-Levin: SAT is first NPC problem. Reductions chain NPC problems together. P=NP? is open problem. Implications: for NPC problems, design approximation algorithms (TSP: Christofides 1.5-approx),

heuristics (simulated annealing, genetic), FPTAS (subset-sum), or exact exponential for small n . Always prove NPC before concluding no efficient algorithm exists.

3. **Q:** Describe how max flow can be used to solve bipartite matching.

Key points: Construct flow network: source $\rightarrow L_i$ (capacity 1), $L_i \rightarrow R_j$ (capacity 1 if edge exists), $R_j \rightarrow \text{sink}$ (capacity 1). Integrality theorem ensures integer flow values. Max-flow = max-cardinality matching. Time: $O(VE)$ via Ford-Fulkerson (each augmenting path adds 1 to matching). Hopcroft-Karp improves to $O(\sqrt{V} \cdot E)$. Weighted bipartite matching $\rightarrow$ min-cost flow or Hungarian algorithm.

4. **Q:** Explain the fork-join parallelism model. What is the significance of work and span?

Key points: spawn = potential parallelism; sync = barrier. Work T_1 = sequential time; Span T_∞ = critical path. Work law: $T_p \geq T_1/P$. Span law: $T_p \geq T_\infty$. Parallelism = T_1/T_∞ (max speedup). Greedy scheduling achieves $T_p \leq T_1/P + T_\infty$. Applications: parallel merge sort (parallelism $\Theta(n/\lg^2 n)$), parallel matrix multiplication. Determinacy races occur when parallel code shares writable data.

5. **Q:** Analyze the performance of hash tables. Compare chaining vs open addressing.

Key points: Chaining: $\alpha = n/m$. Expected search $\Theta(1+\alpha)$. Simple, handles $\alpha > 1$. Open addressing: $\alpha \leq 1$. Expected probes: $\leq 1/(1-\alpha)$ for unsuccessful search. Linear probing: primary clustering, good cache locality. Double hashing: $\Theta(m^2)$ probe sequences, avoids clustering. Universal hashing: guarantees $\Pr[\text{collision}] \leq 1/m$. Practical: choose simple hash function (multiply-shift) and consider memory hierarchy.